



UNIVERSIDAD DE BUENOS AIRES
FACULTAD DE CIENCIAS EXACTAS Y NATURALES
DEPARTAMENTO DE COMPUTACIÓN

Variational Autoencoders para el modelado de estilos de música

Tesis de Licenciatura en Ciencias de la Computación

Lucas Somacal

Directores: Dr. Martín Miguel y Dr. Diego Fernández Slezak
Buenos Aires, 2023

VARIATIONAL AUTOENCODERS PARA EL MODELADO DE ESTILOS DE MÚSICA

En el presente trabajo se abordó el problema de transferencia de estilo en música, es decir, intentar cambiar un fragmento musical de cierto estilo musical a otro. Basándonos en trabajo previo, consideramos la manipulación del espacio latente a partir de un Variational Autoencoder (VAE) con el que codificamos fragmentos musicales a este espacio y operamos mediante vectores característicos de cada estilo musical. En este trabajo, nos propusimos lograr transferencia de estilos entre 4 específicos. A ese fin, comparamos tres modelos. Uno fue entrenado con un *dataset* de música general y luego evaluado en el *dataset* objetivo de 4 estilos. El segundo modelo fue fine-tuneado sobre el *dataset* objetivo y el tercero, solo entrenado sobre este *dataset*. Como parte de este trabajo, también presentamos una metodología de evaluación automática para medir si los fragmentos generados son musicales, se parecen al nuevo estilo y mantienen la identidad del fragmento original. Los tres modelos lograron transformaciones musicales con cambio de estilo. En particular, observamos que la musicalidad y la similitud con el original se van perdiendo a medida que la transformación es mayor pero a su vez se acercan cada vez más al nuevo estilo a medida que crece la magnitud de la transformación, a la vez que el los modelos entrenados sobre el *dataset* mayor obtienen mejores resultados.

Palabras claves: VAE, transferencia de estilos, aprendizaje automático, música, aprendizaje profundo.

VARIATIONAL AUTOENCODERS TO MODEL MUSIC STYLES

In this work, we addressed the style transfer problem in music, that is, trying to change a musical fragment from a certain musical style to another. Based on previous work, we consider the manipulation of latent space from a Variational Autoencoder (VAE) with which we encode musical fragments to this space and operate through characteristic vectors of each musical style. In this work, we aimed to achieve style transfer between 4 specific styles. To that end, we compared three models. One was trained with a dataset of general music and then evaluated in the target dataset of 4 styles. The second model was finetuned on the target dataset and the third, only trained on this dataset. As part of this work, we also present an automatic evaluation methodology to measure whether the fragments generated are musical, resemble the new style and maintain the identity of the original fragment. The three models achieved musical transformations with a change of style. In particular, we observe that musicality and similarity with the original are lost as the transformation is greater but in turn they come closer to the new style as the magnitude of the transformation grows, at the same time the models trained on the greater dataset get better results.

Keywords: VAE, style transfer, machine learning, music, deep learning.

AGRADECIMIENTOS

Cuando a uno le toca agradecer tiene que pensar en responsabilidades y causalidades. Es así que entonces uno se plantea quién o qué es la causa primera, el eslabón inicial de la cadena de hacedores de la presente tesis. Y creo no equivocarme al pensar que lo es la Educación pública, gratuita y de calidad que me ha formado desde los 3 años, en otras palabras, al Estado que con sus enormes falencias siempre estuvo presente. Pero para que esto se dé, es necesario que existan personas que la lleven adelante. Es así que tengo que agradecer en primer lugar a mis directores, especialmente a March que estuvo semana tras semana bancándome y ayudándome en todo; a los docentes que me han formado no solo académicamente, sino también humanamente; a los jurados por el tiempo usado para leer y evaluar la tesis. Sin embargo, no toda la educación formal es vertical. No existiría sin el resto del compañeraje, por lo que menciono especial a CSD, a mi compañero de principio a fin de la carrera y a [8] como así también a toda la buena gente del LIAA. Un honor compartir oficinas con gente tan grossa.

En cuanto a la educación informal (y no menos fundamental), a las personas más importantes, ie, mi familia, papá, mamá, Agus, a la buelica, a Cami y a todo el resto de la familia ampliada. A mis amigos/as/es, de la vida, de coros, de orquestas, de ensambles, de la UBA (secundaria incluida) y de la UNA. En otras palabras, a todas las personas que me han enseñado sobre la vida y me han acompañado.

A la educación pública y gratuita; a mi familia y amistades.

Índice general

1..	Introducción	1
1.1.	Trabajo previo	2
1.2.	Propuesta de trabajo	5
2..	Formalización del problema	6
2.1.	Formato de representación	6
2.2.	Definición de estilo	7
2.3.	Datasets	10
2.3.1.	Análisis sobre el <i>dataset</i>	14
2.4.	Definición del problema	15
3..	Métricas de evaluación	16
3.1.	Estilo	16
3.1.1.	Validación de métrica en D_{Kern}	17
3.1.2.	Evaluación de estilo	17
3.2.	Musicalidad	18
3.2.1.	Formalización	18
3.2.2.	Validación de musicalidad en D_{Kern}	19
3.3.	Similitud con el original	21
4..	Propuesta de Algoritmo Generativo	24
4.1.	Arquitectura	24
4.2.	Propuesta de uso	25
4.3.	Modelos	26
5..	Resultados	31
5.1.	Escucha general	31
5.2.	Musicalidad	32
5.2.1.	Resultados	32
5.3.	Estilo	33
5.3.1.	Resultados	33
5.4.	Similitud al original	35
5.4.1.	Resultados	35
6..	Conclusiones	41
6.1.	Trabajo futuro	42

1. INTRODUCCIÓN

En 1791, el compositor austríaco W. A. Mozart fallece a la edad de 35 años dejando su última obra (la misa de *Requiem* K. 626) inconclusa. Diferentes compositores han presentado posteriormente conclusiones a este trabajo, siendo la más conocida e interpretada la finalización de su único alumno, F. X. Süßmayr (1766 - 1803). En estos últimos años, se han logrado muchos avances en el campo de la Inteligencia Artificial para composición automática de música, obteniéndose resultados notables que pueden confundirse con música creada por humanos [6]. Surge entonces la pregunta: ¿es posible usar estas tecnologías para realizar una nueva continuación de la última obra de Mozart?

Para esta obra en particular, podemos considerar información específica que nos provee el hecho de que sea una misa de *Requiem*. Por un lado, es una misa, es decir, una obra religiosa que está compuesta por diversas partes (como *Kyrie*, *Sanctus* o *Agnus Dei*), con el mismo texto para cada misa. Pero más específicamente, una misa de *Requiem* es una misa de difuntos que consta también de diversas secciones (algunas que son las mismas que las de la ordinaria y otras particulares de este rito) y que al ser de conmemoración de la muerte el carácter es siempre más grave que el de una ordinaria. De este modo, consideramos utilizar estas características para orientar la composición automática a este dominio específico.

En particular, dado que las misas están compuestas por varias secciones que siempre son las mismas, proponemos reconstruirlos en base a dichas partes de otras misas de Mozart y de otros *Requiem*s de otros compositores. Es así que surge la idea de transformar un segmento musical existente en uno nuevo mediante un cambio de estilo. Por ejemplo, tomar el *Sanctus* de una misa ordinaria de Mozart y cambiarle el estilo para que sean el de una misa de *Requiem* o tomar un *Sanctus* de un *Requiem* de otro compositor y cambiarle el estilo para que se asemeje al de Mozart. Esto podemos pensarlo como un problema más general: dado un fragmento de música m , ¿se lo puede transformar para que suene como si fuera escrito en otro estilo estético o con el de otro autor?

Inspirada en esta pregunta, esta tesis se propone el objetivo de probar una técnica de composición automática para *transferencia de estilo*, es decir, intentar cambiar una pieza o fragmento musical de cierto estilo musical a otro. Por ejemplo: dada una canción de The Beatles como *Let it be*: ¿cómo sería si fuera convertida en un tango? En este trabajo, propondremos y evaluaremos un modelo de inteligencia artificial que permita tomar un fragmento de una pieza musical y lo reinterprete en otro estilo. El propósito final es que en el fragmento generado la pieza original sea reconocible y que simultáneamente se reconozca el estilo musical objetivo.

Ahora bien, supongamos que efectivamente construimos un modelo de inteligencia artificial que modifica un fragmento musical para que suene distinto, ¿cómo evaluamos lo generado? ¿Cómo determinamos que la nueva pieza musical *se parece* a la pieza original pero a la vez, al nuevo estilo? En trabajos previos, la evaluación de los modelos suele estar basada en encuestas a personas. Por ejemplo, en el trabajo de MusicVAE [29], se le pregunta al encuestado la preferencia entre fragmentos creados artificialmente o fragmentos originales. De este modo se tiene una noción acerca de la musicalidad de lo generado. Por su lado, en DeepJ [21] también evalúan si el nuevo fragmento es musical, preguntando a oyentes por su preferencia con fragmentos musicales generados por otro modelo de

control. Este modelo permite generar música de algunos estilos predeterminados, por lo tanto, también le preguntan a los encuestados que clasifiquen a qué estilo corresponde el fragmento a evaluar. En otros casos, como en el modelo MuseNet [25], no se presenta ningún tipo de evaluación. En conclusión, la solución común suele ser hacer encuestas y no hay una metodología automática.

Como parte de este trabajo, nos propusimos desarrollar metodologías automáticas para capturar la calidad de los fragmentos musicales generados por el modelo de inteligencia artificial presentado. En particular, decidimos enfocarnos en los aspectos de la evaluación de los trabajos previos: determinar si el nuevo fragmento pertenece al nuevo estilo y determinar la musicalidad del fragmento generado.

Además, proponemos un tercer eje de evaluación que es la similitud con el fragmento original. Si no lo consideramos, podríamos estar evaluando positivamente un fragmento generado como muy musical y en el estilo buscado pero que nada tiene que ver con el fragmento original, siendo en rigor una transferencia de estilo insatisfactoria.

En conclusión, en este trabajo abordaremos dos problemas: proponer un modelo de inteligencia artificial que permita modificar un fragmento musical a un estilo deseado y definir una metodología formal de evaluación de los fragmentos generados.

A continuación presentamos qué se ha hecho previamente en el dominio de transformar obras artísticas a un estilo distinto del original, tanto en otros dominios como en el de la música. Luego detallaremos los avances propuestos por este trabajo en el marco del trabajo previo.

1.1. Trabajo previo

La transferencia de estilo (*style transfer*) consiste en, dado un input - que puede ser una imagen, un texto, un fragmento musical - cambiarlo para que presente ciertas características nuevas, pero preservando la identidad original.

Este es un problema ya abordado en otras áreas como imágenes o texto [16]. Por ejemplo, el trabajo de Gatys et al. [11] busca modificar imágenes para un propósito en particular: dada una imagen original y una imagen de referencia de estilo, crear una imagen nueva a semejanza del original, pero tomando características de la referencia. En la Figura 1.1 se observa cómo cambian una foto para asemejarla con una acuarela de Turner.



Fig. 1.1: Imagen transformada a estilo “acuarela de Turner”. A) Imagen original. B) Imagen transformada e imagen “característica” del estilo. Figura obtenida de [11].

Con propósitos similares, Hou et al. [15] logra transformar imágenes con el objetivo de agregarle o quitarle alguna característica a la imagen original. Así, por ejemplo, como se ve en la Figura 1.2, se le agregan o quitan sonrisas o anteojos a distintas imágenes originales de caras.

Otra familia de modelos son los que trabajan a partir de *prompts*, indicaciones en texto natural describiendo el contenido y características estilísticas de la imagen. Este tipo de modelos en general se utilizan para generar imágenes utilizando únicamente el *prompt* como entrada [22, 24]. Algunos de estos modelos permiten tomar una imagen de referencia e indicar mediante texto cómo queremos que la misma sea modificada [32].

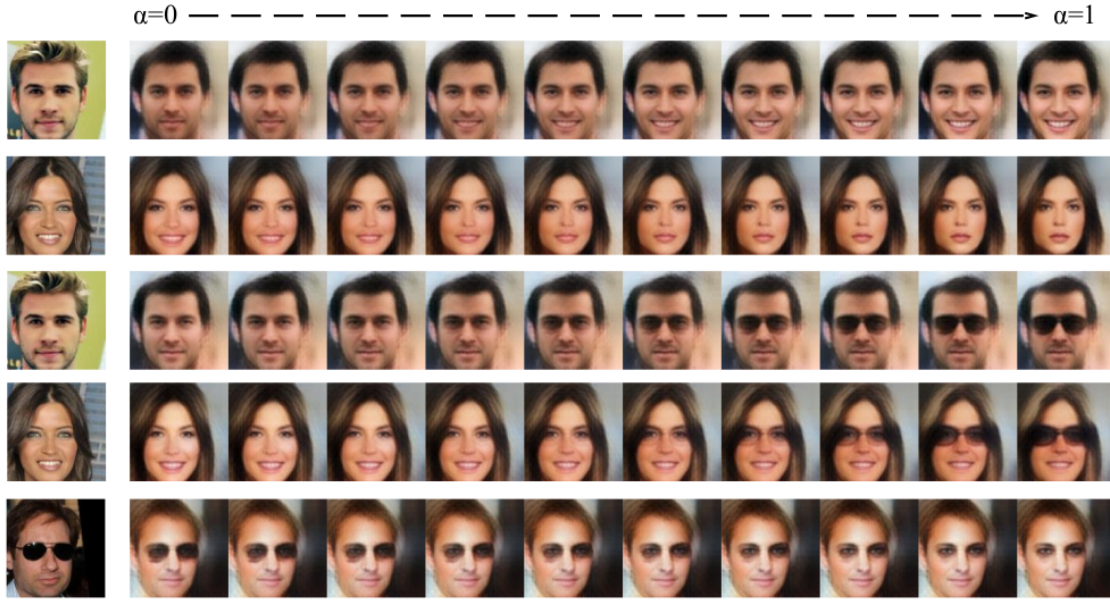


Fig. 1.2: Ejemplo de generación de imágenes mediante manipulación del espacio latente. Imagen extraída de [15].

En el dominio de la música, hay mucho trabajo realizado en cuanto a composición automática [3, 4] (para más referencias consultar el libro de Briot et al. [6]). La música se puede representar en su forma más completa como una grabación de audio. Sin embargo, en la mayoría de los casos los modelos trabajan con representaciones simbólicas (que solo tienen información de cuáles sonidos deben sonar en qué momento). Estas representaciones luego pueden ser convertidas en audio.

Dentro del subconjunto de modelos que toman un fragmento musical como entrada, hay diversos problemas ya tratados. Por ejemplo, hay sistemas que producen la melodía y la armonía al mismo tiempo dada una semilla inicial (una nota o conjunto de notas), como por ejemplo, DeepHear [33]. Otros sistemas tratan de producir música a partir de algún tipo de condicionamiento más particular. Por ejemplo, Minibach genera el acompañamiento a partir de una melodía (sección 6.2.2 de [6]); el sistema propuesto por Makris et al. genera secuencias de ritmos a partir de estructuras rítmicas base [20]; MidiNet crea melodías en base a acordes [36]; MusicVAE genera melodías en base a melodías previas pero considerando su estructura general y pequeños fragmentos dentro de esta como condicionante principal [29]. También pueden encontrarse sistemas que crean música condicionados al género musical o al instrumento. En particular, hay modelos que permiten condicionar por

estilo, como DeepJ [21] o MuseNet [25].

DeepJ es un modelo generativo *end-to-end* basado en LSTMs capaz de componer música condicionado a varios estilos de compositores posibles. Pueden así aprender el estilo musical y generar nueva música polifónica dada una secuencia de notas iniciales. Para modelar el estilo consideran el conjunto de todos los compositores del *dataset* utilizado y los representan con *one-hot encoding*. En particular, en este modelo, los datos de entrenamiento y la música generada puede considerarse una mezcla de estilos. Así, por ejemplo, si el estilo de interés está formado por los compositores 1 y 3, el vector que lo representa será $[0,5; 0; 0,5; 0...]$. Finalmente, este vector puede ser usado como input condicionante de la música que producirá el sistema modelado como embedding [17].

MuseNet [25], por su lado, usa *transformers* para componer hasta 4 minutos de música en un determinado estilo. Dada una secuencia inicial de notas y un estilo a imitar, compone nueva música. Para ello, MuseNet se basa en GPT-2, un modelo de lenguaje. Para usarlo, se convierten los MIDI de entrada en texto y se agregan *tokens* de contexto (estilo, autor) al inicio. En particular, MuseNet usa *kernels* optimizados de *Sparse Transformer* [9] para entrenar una red de 72 capas con 24 *attention heads* (con *full attention*) sobre un contexto de 4096 *tokens*. De este modo *recuerda* la estructura de la pieza y con los *tokens* iniciales condiciona el estilo.

En ambos casos la generación es desde una semilla a partir de la cual genera nueva música que la continúa, por lo que no nos sirve para resolver nuestro problema donde ya partimos de un fragmento musical dado y lo que queremos es que lo que se transforme esa música sobre la base de ese mismo fragmento, no que se continúe a partir de lo escrito.

Finalmente, con el advenimiento de los *Large Language Models*, han surgido sistemas como MusicLM [1] que buscan crear audios de música en base a instrucciones escritas en lenguaje natural o en base a un audio de música y un condicionamiento expresado en inglés. Este modelo resuelve las tareas de: generación de audio a partir descripciones generales sobre el estilo, a partir de descripciones para instantes de tiempo particulares, a partir de un audio de una melodía silbada, generación de un audio de dicha melodía con otros instrumentos, y generación de audio a partir de una imagen. Sin embargo, tampoco es posible especificarle, dado un fragmento musical, que componga algo en un estilo musical particular.

AudioGPT [2] intenta resolver diversas tareas de generación de audio a partir de texto. En particular, entre las tareas relacionadas con música, el modelo solo permite generar audio de una voz cantada a partir de una secuencia de notas. Asimismo, genera audios a partir de imágenes. Pero en todo caso, tampoco permite resolver el problema que planteamos.

Por otro lado, en text2music [35] generan música simbólica a partir de texto en lenguaje natural pero específicamente aclaran que no es posible hacer *style transfer*. Puede reproducir melodías presentes en el *dataset* de entrenamiento, puede componer arreglos de melodías, y extraer información como la tonalidad de un fragmento musical dado.

En conclusión, no encontramos en la bibliografía ningún modelo que permita transformar música en base a alguna música previa y a un estilo musical dados. En el presente trabajo proponemos un modelo de inteligencia artificial para este problema.

1.2. Propuesta de trabajo

En el presente trabajo se abordará una propuesta de resolución del problema de *style transfer* para fragmentos musicales a partir de un modelo de aprendizaje automático y una propuesta de evaluación automática de este. En el siguiente capítulo, comenzamos por abordar la definición formal del problema, que conlleva definir el formato de representación del fragmento musical, las características necesarias de los *datasets* a usar, elegir un *dataset* específico dadas las definiciones previas, formalizar la noción de estilo y de la tarea de transferencia de estilos.

En el capítulo 3 se plantearán las preguntas que queremos responder para evaluar los modelos propuestos. Estas preguntas llevarán a definir las métricas de evaluación. Abordaremos cómo determinamos si un fragmento musical pertenece a un estilo, formalizaremos una noción de musicalidad, y si un fragmento transformado mantiene semejanzas con el fragmento original. Además, a cada una de estas métricas las validamos en el *dataset* usado.

En el capítulo 4 detallamos la arquitectura utilizada para nuestro modelo. Esta se basa en *Variational Autoencoders* (VAE), una arquitectura particular de redes neuronales, por lo que detallaremos cómo funcionan y cómo las utilizaremos para nuestro problema. Además, se desarrollará sobre los distintos modelos entrenados y los detalles del proceso de entrenamiento. En particular, se entrenaron 3 modelos. El primero, sobre el *dataset* objetivo, el segundo sobre un *dataset* más grande y general, y el tercero, el resultado de aplicar *fine-tuning* al modelo anterior sobre el *dataset* objetivo.

Luego, en el capítulo 5 presentaremos y analizaremos los resultados de evaluar con las métricas del capítulo 3 cada uno de los modelos así como un análisis informal tras escuchar un subconjunto de audios generados.

Finalmente, en el último capítulo evaluamos los logros y limitaciones del trabajo realizado, así como también proponemos futuras direcciones de investigación.

2. FORMALIZACIÓN DEL PROBLEMA

Para poder plantear el problema, se deben considerar los siguientes puntos:

- Cómo representar a los fragmentos musicales.
- Cómo se define formalmente a un estilo musical.
- Qué dataset usar.
- Definir formalmente el problema.

A continuación se detallarán estos 4 puntos.

2.1. Formato de representación

La primera cuestión a resolver es cómo representamos a un fragmento musical m . Para representar música en general hay dos paradigmas: representación de audio y simbólica (que consiste en especificar cuánto es la duración de una nota, cuándo empieza y qué altura tiene, considerando a estas variables como atributos fijos de una nota). Esta separación se corresponde a pensar el problema de la música como variables continuas o discretas, respectivamente [6]. Si bien el formato de audio se corresponde con lo que percibimos como música, es decir, tiene las cualidades del sonido, no evidencia las cualidades discretas que percibimos, como las duraciones o las alturas de cada nota en particular. En este trabajo decidimos utilizar la representación simbólica ya que refiere a características cognitivas de la música y existe más trabajo de referencia en este formato.

En principio, podemos pensar a m como una serie de notas que tocan un grupo de instrumentos. En este trabajo vamos a simplificar dicha representación musical. Dado que normalmente una pieza musical suele tener más de dos instrumentos, podemos simplificarlos a dos más abstractos: una línea melódica y una de bajo. En esta simplificación, no estamos permitiendo notas simultáneas (acordes) en cada línea. Esta simplificación funciona en general bien para describir la mayor parte de la música occidental. A estas abstracciones las llamamos *tracks*. Más formalmente, un *track* será una secuencia de notas o silencios (cuando no hay ninguna nota) de duraciones proporcionales a una unidad base de tiempo. En particular, se utiliza una unidad de tiempo de referencia ya que permite tocar la misma pieza más rápido o más lento, cambiando la duración de esta unidad base pero manteniendo las duraciones relativas entre notas (o silencios). Esta es la manera en la que se suele notar la música occidental.

De este modo, para representar a cada *track* vamos a usar un formato matricial basado en un *piano roll*. Un *piano roll* es una representación bi-dimensional de la información simbólica de un fragmento musical comúnmente utilizado en programas de composición. Para nuestro modelo, utilizamos una matriz donde las filas representan una unidad de tiempo y las columnas las alturas de la nota. En esta matriz hay un 1 si una nota suena en un determinado momento. A esto debemos agregar una columna que indica si ninguna nota suena en ese instante (silencio). De este modo, dado que un *track* es una secuencia de notas y silencios (recordemos que no se permiten notas simultáneas en cada línea), esta matriz tendría por fila exactamente solo un 1.

Adicionalmente, se le agrega una última columna que indica el inicio de una nueva nota (ritmo). Esta información es redundante cuando hay un cambio de nota pero es necesaria para identificar la repetición de notas. En la Figura 2.1 se puede ver un ejemplo de una melodía familiar pero simple: el *Feliz cumpleaños* (o *El Payaso Plim Plim*). Se puede ver que empieza con 2 notas iguales que duran 3 y 1 unidades de tiempo respectivamente. Se sabe que son distintas notas por la última columna. Luego, un tono arriba hay una nota que dura 4 unidades de tiempo, después se baja 1 tono, etc. Al final del ejemplo se tiene un caso de silencio que dura 4 unidades.

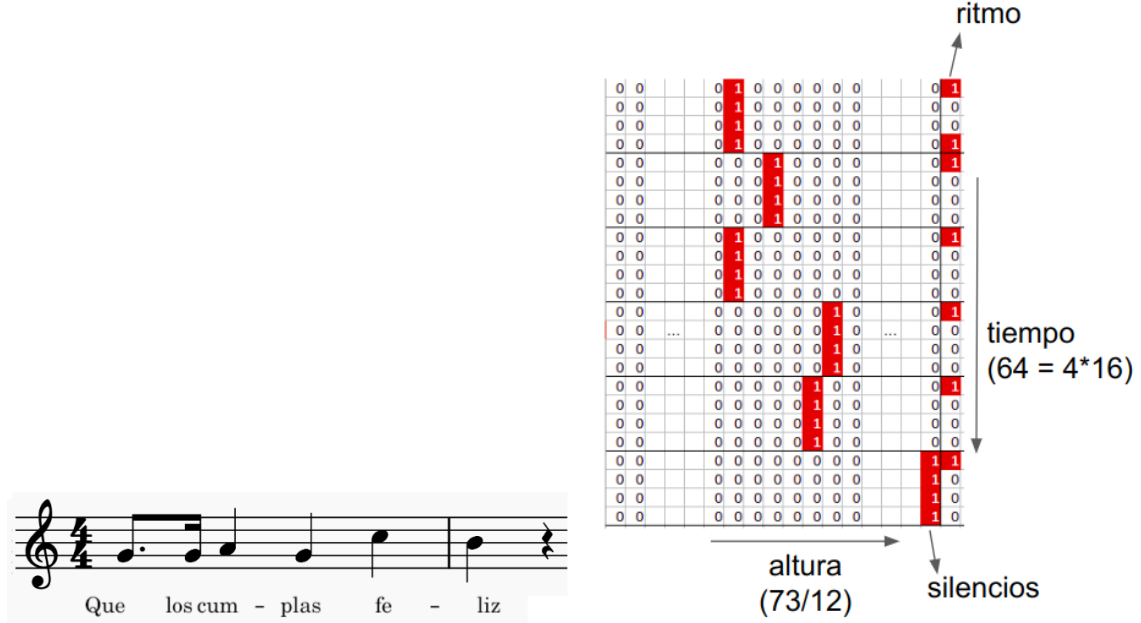


Fig. 2.1: Primeras notas del *Feliz cumpleaños* (o *El payaso Plim Plim*). A la izquierda, la partitura. A la derecha el *piano roll* modificado.

El *piano roll* extendido nos permite representar un *track*. Por lo que para representar m , simplemente se concatenan ambas matrices, una para la melodía y otra para el bajo. En particular, el *track* de la melodía tendrá 73 notas posibles (es decir, tiene dimensión 75) y el bajo tiene 12 (dimensión 14). Para el bajo podemos utilizar solo 12 notas ya que su función suele indicar la armonía que forma junto con la melodía, siendo importante qué nota es y no a qué altura (en qué octava) se encuentra [12]. Además, ambos *tracks* se encuentran desarrollados en 64 unidades de tiempo (4 compases de 16 semicorcheas cada uno). Esto conlleva a que la dimensión final de la matriz será de 64×89 .

2.2. Definición de estilo

La siguiente pregunta que se plantea es: ¿qué es un estilo? Para responder esta pregunta, pensemos en lo que coloquialmente definimos como estilo musical. Si hablamos de un carnavalito nos remitimos a canciones con un ritmo e instrumentos característicos o, si hablamos de Gardel, pensamos inmediatamente en tangos. Se pueden entonces pensar a los estilos como un conjunto de piezas que comparten características en común. Estas características pueden ser ritmos, autores, períodos, estéticas, etc. Por otro lado, las características surgen a veces a la inversa, es decir, a partir de un conjunto de piezas con algún

factor común que las identifique (como por ejemplo, las composiciones de Ástor Piazzolla que no se las consideró ni como tango ni como música académica, pero en su conjunto generaron un nuevo estilo). Por lo tanto, como definición práctica tomaremos como estilo a un conjunto de piezas con alguna característica en común. Si se toma como característica el autor, podemos pensar, por ejemplo, en el *estilo Bach* o el *estilo Mozart*. En particular, dado un *dataset* de piezas de determinados autores, podemos centrarnos en las piezas de cada uno de ellos para definir cada estilo. De este modo, dado un fragmento de una pieza del *dataset* lo que buscaremos es modificarlo para que se parezca a otro de los estilos del *dataset*.

Necesitamos entonces alguna forma de caracterizar el estilo a partir de un conjunto de piezas para luego medir y determinar la pertenencia o no a un estilo. Para ello, consideramos el trabajo de Rodríguez Zivic et al. [30] en el que distingue composiciones de distintos períodos históricos musicales en función de la distribución de sucesiones de 2 intervalos melódicos. Un intervalo melódico es la distancia en altura que hay entre dos notas sucesivas. En la mayoría de los sistemas tonales, y en particular en el occidental, las distancias entre alturas de notas se miden en *semitonos*.

Entonces, podemos considerar a una melodía como una sucesión de intervalos melódicos. Luego, para definir la estética de la melodía nos podemos preguntar, dado un intervalo x (con $x \in \{-12, \dots, 12\}$ semitonos), ¿cuál es la probabilidad de que el siguiente intervalo sea y (con $y \in \{-12, \dots, 12\}$, la distancia en semitonos entre una nota y la otra¹)? La hipótesis que tomamos a raíz del trabajo de Rodríguez Zivic et al. es que esta distribución de probabilidad es una descripción útil del estilo.

Para calcular estas distribuciones, calculamos para cada subconjunto de fragmentos M la matriz $\Sigma^i(M)$ de 25×25 donde cada elemento Σ_{xy}^i es la cantidad de ocurrencias del intervalo y que se suceden luego de un intervalo x en todos los fragmentos de M .

En la Figura 2.2 se pueden observar dichas distribuciones de intervalos para los cuatro estilos que utilizaremos en este trabajo (ver sección 2.3). El eje x representa la cantidad de semitonos entre dos notas y el eje y , la cantidad entre la segunda nota del intervalo del eje x y la siguiente nota, teniendo en el centro al 0 (no cambia la nota). Se puede observar que el estilo de Bach es el menos disperso, concentrándose los bigramas de intervalos en unos pocos en particular y de poca magnitud. El estilo de Frescobaldi es similar al de Bach pero con un poco más de dispersión. Notemos que ambos pertenecen al mismo período estético (el barroco). El estilo de Mozart, por su parte es más disperso pero siguen teniendo importancia los mismos bigramas de intervalos preponderantes en Bach. Finalmente, el estilo más distinto es el de los ragtimes que no solo presenta mayor dispersión, sino que los bigramas principales son distintos que los de los otros estilos.

A su vez, análogamente, proponemos caracterizar un estilo en función de la distribución de bigramas de patrones rítmicos. En particular, consideramos los patrones posibles que se pueden dar en cuatro unidades de tiempo. Si notamos con un 1 cuando comienza una nueva nota (de distinta o misma altura) y con un 0 cuando no lo hay, tenemos 16 posibles patrones dados por las combinaciones de unos y ceros en arreglos de 4 unidades. Por ejemplo, el patrón de que hay solo cambio de nota en las dos primeras unidades será 1100. Así obtenemos una representación binaria de patrones rítmicos. En particular, podemos pensar a estos patrones como números en base binaria que al transformarlos a base decimal se obtienen los números enteros entre 0 y 15.

¹ Dado que los nombres de las notas se repiten cada 12 semitonos, podemos pensar a los intervalos como la distancia en semitonos módulo 12, al que se le añade el signo para indicar dirección.

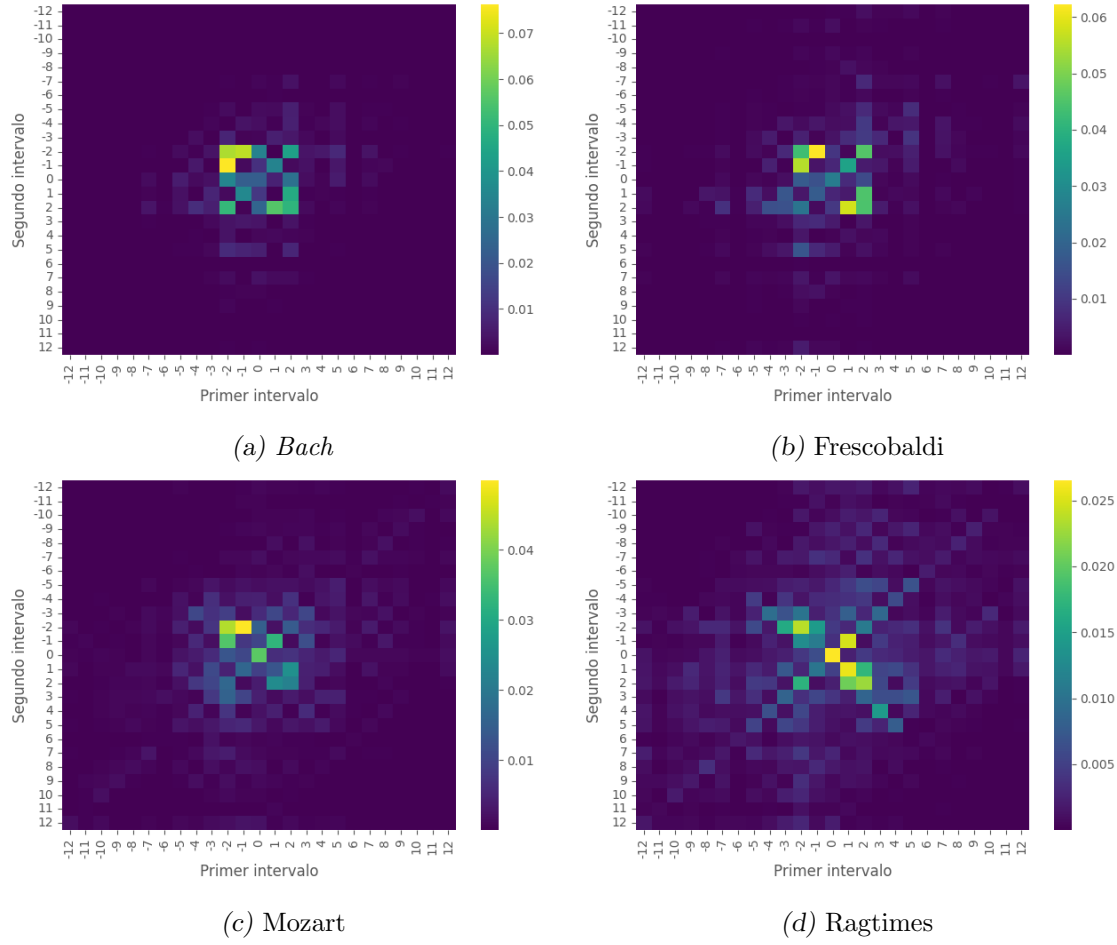


Fig. 2.2: Distribución de bigramas de intervalos para cada estilo. El eje x representa la cantidad de semitonos entre dos notas y el eje y , la cantidad entre la segunda nota del intervalo del eje x y la siguiente nota.

De forma análoga al caso de los intervalos, para calcular las distribuciones consideramos para cada subconjunto de fragmentos M la matriz $\Sigma^r(M)$ de 16×16 donde cada elemento Σ_{ij}^r es la cantidad de veces que el patrón rítmico j es precedido por el patrón rítmico i .

De este modo, en la Figura 2.3 podemos observar las distribuciones de bigramas patrones rítmicos para los 4 estilos trabajados donde en cada gráfico el eje x representa el primer patrón rítmico del bigrama y el eje y el segundo. Se puede observar que el estilo de Bach está caracterizado principalmente por la sucesión de patrones 8-8, es decir 1000-1000 con muy pocas apariciones de otros patrones. En contraposición, si bien el estilo de Frescobaldi presenta sucesiones 8-8, el bigrama dominante es 0-0, es decir, pocos cambios de notas. Por otra parte, Mozart y ragtime tienen como bigrama dominante al 16-16 (o sea, 1111-1111), es decir, un cambio de nota en cada unidad de tiempo. Sin embargo, los ragtimes presentan muchos otros bigramas de patrones que Mozart.

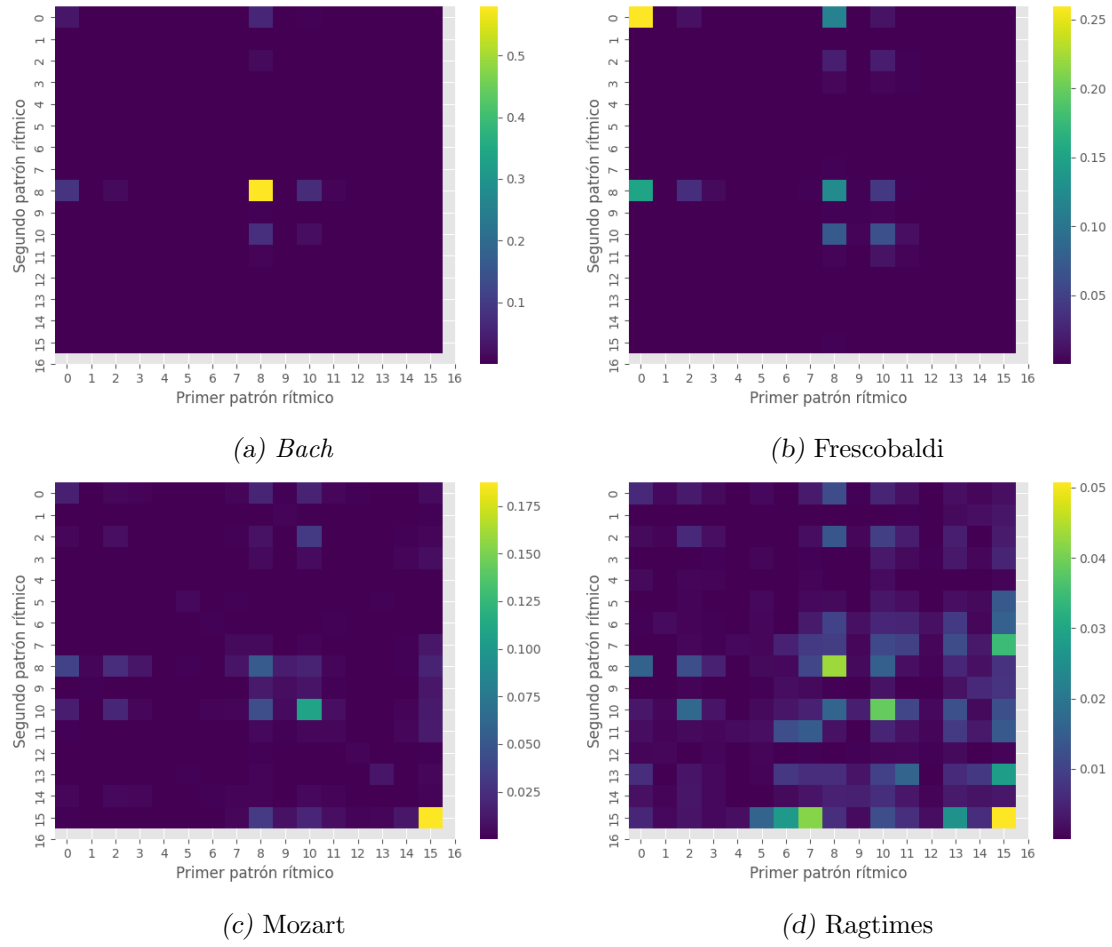


Fig. 2.3: Distribución de bigramas de ritmos para cada estilo

2.3. Datasets

Considerando las definiciones de las secciones anteriores, surge la necesidad de buscar un *dataset* en formato simbólico que contenga distintos estilos. En particular, hay distintos formatos digitales de representación de música simbólica, entre los que citamos el MIDI, el piano roll, MusicXML y ABC [6]. En todos estos, hay información de notas y duraciones, por lo que podemos convertirlo al formato final que vamos a usar.

Sería posible usar audios pero implicaría agregar un proceso de traducción a formato simbólico. Esto agrega complejidades y la posibilidad de introducir errores, por ejemplo, en la definición de las notas, cuándo empiezan y cuándo terminan o la separación de los distintos instrumentos. Por otro lado, para nuestro problema los datos deben estar cuantizados, es decir, la ubicación y duración de las notas se encuentran en una grilla fija de valores iguales, siendo estos un múltiplo de una unidad básica de tiempo. De este modo, se optó por buscar directamente datasets en formato simbólico cuantizado desde su creación.

Por otro lado, debe haber metadatos de la pieza musical asociada al estilo (o al autor) y debe contener la suficiente cantidad de piezas para poder ser usada como entrenamiento

de un algoritmo de aprendizaje automático. Además, en particular, se buscó que en lo posible contuvieran suficientes datos de música académica, estilo al que pertenece Mozart, y de estilos más y menos similares para facilitar el entrenamiento de la tarea. Finalmente priorizamos aquellos que tuvieran acceso libre.

A partir de una búsqueda de posibles datasets de música en formato simbólico encontramos el siguiente conjunto:

- *Classical archives*²: dataset con 20 mil MIDIs y más de 800 mil archivos de audio de música clásica de diversos autores y estilos (dentro de ese tipo de música). El acceso no es gratuito.
- *Digitalen Mozart Edition*³ (DME): contiene MIDIs de un único autor (Mozart). Podría ser usado para obtener un estilo pero aunque está pensado como repositorio de acceso público, no permite descargar los datos de forma sencilla, por lo que debería hacerse *scraping*.
- MAESTRO⁴: contiene 200 horas de MIDIs con metadatos pero fueron generados a partir de la ejecución reales de pianistas en concursos, por lo que no se encuentra cuantizado y solo contienen obras para piano de música clásica.
- *International Music Score Library Project* (IMSLP)⁵: es un dataset de música clásica de 222 mil obras sin derecho de autor. También al igual que DME está pensada como biblioteca y hay que hacer *scraping* de todo el catálogo para obtener estadística de qué datos simbólicos tiene.
- *Lakh Midi Dataset*⁶: Contiene más de 176000 MIDIs aunque no todos tienen metadatos de géneros. Solo poco más de 45000 tienen metadatos (aunque no necesariamente de géneros). Es de acceso gratuito y está disponible para bajarlo todo en su conjunto.
- *Kernscores*⁷: Contiene 108 mil archivos en distintos formatos simbólicos de distintos géneros, principalmente música clásica. Están categorizados por autor y por estilo cuando no es música clásica. Se lo puede bajar en su conjunto con un sencillo *scraping* de la página.
- *Kunstderfuge*⁸: dataset con más de 20 mil MIDIs de música clásica categorizados por autor. De forma gratuita permite un máximo de descarga de 5 archivos por día.
- *Nottingham*⁹: dataset con más de 1000 canciones en formato ABC de un único género: folk. Contiene solo la melodía. Es de acceso libre.
- *music21*¹⁰: contiene alrededor de 3000 piezas de distintos autores, principalmente de música clásica pero también de otros géneros como el folk. Se encuentran en distintos formatos, todos de ellos simbólicos. Es de acceso libre.

² www.classicalarchives.com/

³ <https://dme.mozarteum.at/musik/edition/>

⁴ <https://magenta.tensorflow.org/datasets/maestro>

⁵ <https://imslp.org>

⁶ <https://colinraffel.com/projects/lmd/>

⁷ <https://kern.humdrum.org/>

⁸ <http://kunstderfuge.com/>

⁹ <http://abc.sourceforge.net/NMD/>

¹⁰ <https://web.mit.edu/music21/doc/about/referenceCorpus.html>

- *8notes*¹¹: casi 50 mil partituras de distintos estilos. No tiene límite de descargas y contiene metadatos de autor pero se debe hacer *scraping* para descargar en conjunto.

Otros datasets considerados de características similares fueron:

- *GiantMIDI*¹² [19]: más de 10 mil MIDI's de composiciones clásicas para piano de los cuales más de 7 mil se encuentran anotadas con autor. El acceso es libre.
- *Classical music midi* (dataset disponible en *kaggle*)¹³: 300 MIDI's de música clásica clasificados por autor, de acceso libre.
- *Symbolic Music Dataset*¹⁴ [34]: 20 mil MIDI's de acceso libre de distintos géneros. Carece de metadatos.
- *TheoryTab database*¹⁵: más de 40 mil piezas, contiene metadatos de autor pero se debe hacer *scraping* para descargar todo en conjunto.

Dada esta disponibilidad de *datasets*, nos enfocamos en aquellos con las características deseadas. En particular, decidimos utilizar *Kernscores* que tiene distintos autores clásicos y no clásicos con distintos niveles de similitud. Comenzamos trabajando con dos estilos iniciales: corales de Bach¹⁶ y ragtimes¹⁷ tomados de la base de *Kernscores*. Los corales de Bach consisten en melodías (y bajo) fuertemente homorrítmicas¹⁸ y con ritmos sencillos. Melódicamente se caracterizan por tener pocos saltos o si los hay, están compensados¹⁹. Los ragtimes presentan saltos más grandes, registro más amplio, células rítmicas que se repiten e independencia entre bajo y melodía. De esta manera, observamos que presentan características musicales marcadamente distintas y fácilmente reconocibles auditivamente como para poder evaluar sencillamente los resultados iniciales.

De este modo, hicimos una primera implementación del modelo (ver capítulo 4) de cambio de estilo y notamos que al transformar un fragmento de Bach en ragtime se produjeron más saltos en la melodía y que el ritmo en general se desemparejó. Frente a estos resultados, nos preguntamos si estas transformaciones estaban efectivamente asociadas a un cambio de estilo o era un resultado de la aleatoriedad generada por el modelo. Con lo cual, decidimos usar otros dos *datasets* de estilos más parecidos a Bach para comparar el comportamiento entre ellos y contra los otros 2 estilos.

Estos dos *datasets* son de sonatas para piano de Mozart y canciones de Frescobaldi²⁰ también tomados de *Kernscores*. Son más simples melódicamente que las de ragtime y el bajo y la melodía son relativamente independientes rítmicamente. En el caso de Mozart, uno de los *tracks* suele ser el acompañamiento de la otra (en general el bajo acompaña) y suele corresponder a una repetición de figuras iguales con pequeñas variaciones de notas. En Frescobaldi, en cambio suelen ser dos melodías que evolucionan por su lado (o con

¹¹ <http://www.8notes.com/>

¹² <https://github.com/bytedance/GiantMIDI-Piano>

¹³ <https://www.kaggle.com/soumikrakshit/classical-music-midi>

¹⁴ <https://github.com/wayne391/Symbolic-Musical-Datasets>

¹⁵ <https://www.hooktheory.com/theorytab>

¹⁶ Compositor barroco alemán (1685 - 1750).

¹⁷ Baile de origen afroamericano de finales del siglo XIX y principios del XX.

¹⁸ Es decir, con ritmos iguales entre melodía y bajo.

¹⁹ Es decir, si el salto es muy grande, el siguiente será en sentido contrario.

²⁰ Compositor barroco italiano (1583 - 1643).

grupos de notas similares pero en distinto momento). Se espera que las transformaciones que tengan como objetivo a Mozart o a ragtime sean más vivas rítmicamente, mientras que hacia Bach se acomoden en ritmos más pausados y uniformes. Melódicamente también se esperarían intervalos más grandes y diversos al transformar a ragtime y lo contrario al transformar a Bach.

Se tomaron 166 corales de Bach, 39 canciones de Frescobaldi, 52 movimientos de sonatas para piano de Mozart y 37 ragtimes. Sin embargo, la longitud de cada una es bastante distinta, y particularmente entre estilos: los corales duran alrededor de 20 compases mientras que los ragtimes alrededor de 150. Con lo cual, la cantidad de fragmentos de 4 compases no es proporcional a la cantidad de piezas de cada estilo. En la tabla 2.1 se detallan las cantidades de piezas y fragmentos para cada estilo, obteniéndose un total de 3850 fragmentos.

Estilo	Piezas	Fragmentos
Bach	166	508
Frescobaldi	39	661
Mozart	52	1309
Ragtimes	37	1372

Tab. 2.1: Cantidad de piezas y fragmentos para cada estilo del *dataset* D_{Kern}

A partir de este conjunto de datos (al que denominamos D_{Kern}), se procedió a entrenar el primer modelo (que explicaremos en el capítulo 4). A partir de escuchar algunos de los fragmentos producidos se observó que parecían ser poco musicales puesto que las transformaciones divergían rápidamente en notas aparentemente aleatorias. Surgió entonces la hipótesis de que eran pocos los fragmentos usados para entrenar. Dado que en Aprendizaje Profundo es frecuente entrenar un modelo sobre un *dataset* amplio que luego se adapta para dominios específicos, proponemos entrenar sobre un nuevo conjunto de datos, *Lakh MIDI Dataset* [28] (*dataset* que llamaremos D_{LMD}) para luego evaluarlo sobre D_{Kern} . Este proceso es detallado en el capítulo 4.

El *Lakh Midi Dataset* cuenta, entre otra información, con metadatos asociados al estilo de cada pieza. En particular, fueron de interés los `mb_tags`, etiquetas de estilo de la base digital `musicbrainz.org`. Considerando estos *tags*, se eligieron 4 de los que contenían por lo menos 500 piezas. En la tabla 2.2 se detallan la cantidad de piezas etiquetadas con cada uno de los 4 *tags* elegidos.

Estilo	Piezas
classic pop and rock	4009
pop	3445
folk	791
classical	593

Tab. 2.2: Cantidad de piezas para cada estilo del *dataset* D_{LMD}

Cabe aclarar que cada pieza tiene varios *tags*, por lo que hay piezas que pueden tener más de uno de estos. De este modo, luego de preprocesarlos, se obtuvieron en total 155.037 fragmentos de 4 compases (es decir, un *dataset* 40 veces más grande que el anterior).

2.3.1. Análisis sobre el *dataset*

Decidimos analizar formalmente las hipótesis que presentamos sobre los cuatro estilos de D_{kern} sobre los que evaluaremos nuestros modelos en la tarea de transferencia de estilo. Por un lado, los estilos de Bach y Frescobaldi son los más sencillos melódica y rítmicamente mientras que Mozart y los ragtimes son más complejos y presentan mayor diversidad de patrones rítmicos y de saltos, siendo los ragtimes los más complejos. Para formalizar estas descripciones calculamos la complejidad rítmica y melódica del conjunto de fragmentos de cada estilo.

Es así que surge la idea de reutilizar nuevamente el trabajo de Rodríguez Zivic et al. donde se plantea la hipótesis de que esta distribución de probabilidad depende del estilo. Calculamos entonces las distribuciones de bigramas de intervalos y de patrones rítmicos para cada estilo como explicamos en 2.2. Como noción de complejidad tomamos la entropía de Shannon [31]:

$$H = - \sum (p \cdot \log(p)) \quad (2.1)$$

donde p es la probabilidad de ocurrencia de una nota dadas las dos anteriores. En particular, calculamos $H(\Sigma^i(M))$ y $H(\Sigma^r(M))$ para representar complejidad melódica y rítmica, respectivamente, tomando a M como los fragmentos de los 4 estilos mencionados.

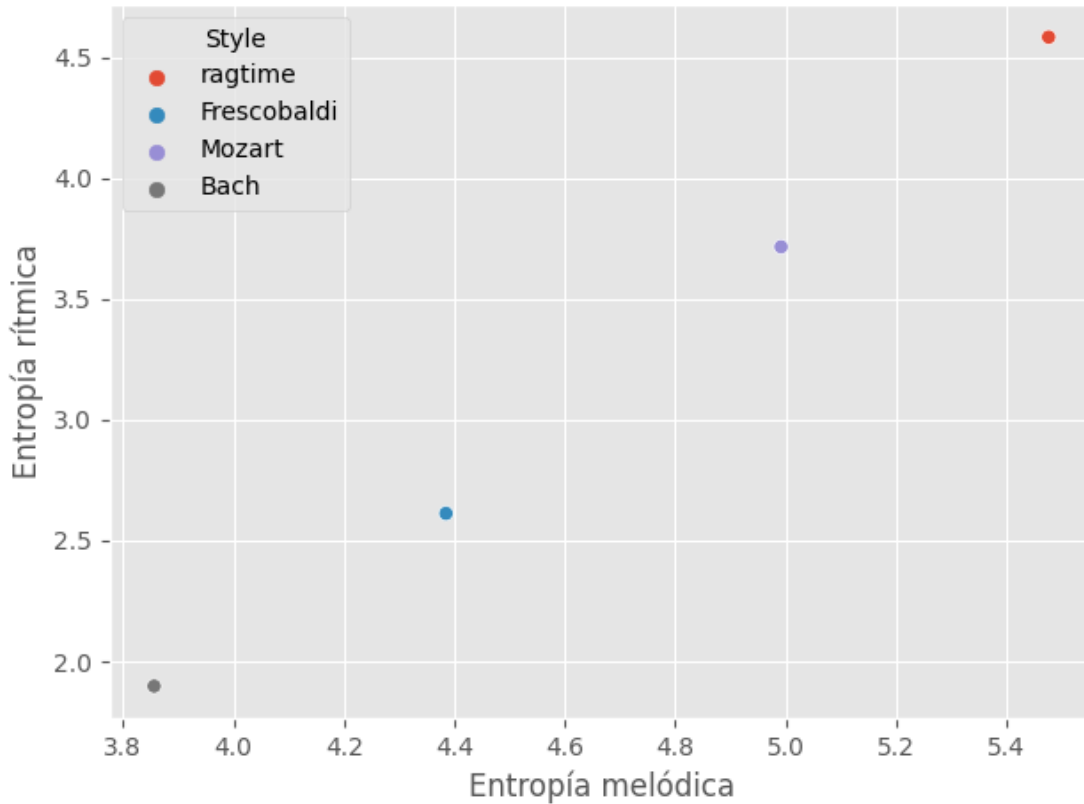


Fig. 2.4: Entropías de melodía (eje x) y ritmo (eje y) para los estilos Bach (gris), ragtime (rojo), Mozart (lila) y Frescobaldi (celeste).

En la Figura 2.4 se puede observar el resultado de calcularle esta medida de complejidad

a cada uno de los *subdatasets*. En el eje x figura la entropía respecto a las alturas de los sonidos y en el eje y respecto a las duraciones. Efectivamente podemos comprobar que Bach y ragtime aparecen en extremos opuestos de complejidad y Mozart y Frescobaldi en lugares intermedios, siendo este el más simple de los 2. En otras palabras, hay una diferencia de complejidad entre cada uno de los estilos y se corresponde con la planteada como hipótesis.

En ambos casos es apreciable que la complejidad de cada gráfico se corresponde con las complejidades reportadas con la entropía.

2.4. Definición del problema

Recordemos que el problema que queremos resolver es que dado un fragmento musical sea reinterpretado en otro estilo. Para formalizarlo, consideremos un dataset D de fragmentos musicales y dos subconjuntos M_0 y M_1 de estilos s_0 y s_1 respectivamente. Notemos que s_0 no es exclusivamente el conjunto M_0 sino el conjunto de todos los fragmentos musicales posibles que podrían ser categorizados como s_0 , lo cual incluye a M_0 . Lo que queremos encontrar es alguna función $t : s_0 \rightarrow s_1$ tal que $t(m) = m'$ con m' un fragmento nuevo similar a m pero cuyo estilo se asemeje a s_1 . Algo que hay que notar es que el m' que se genera no necesariamente está en D . En particular, la idea es que pertenezca a $s_1 \setminus M_1$. Es decir, que no genere simplemente un fragmento ya existente del estilo objetivo. En la Figura 2.5 ilustramos el problema con diagramas de Venn.

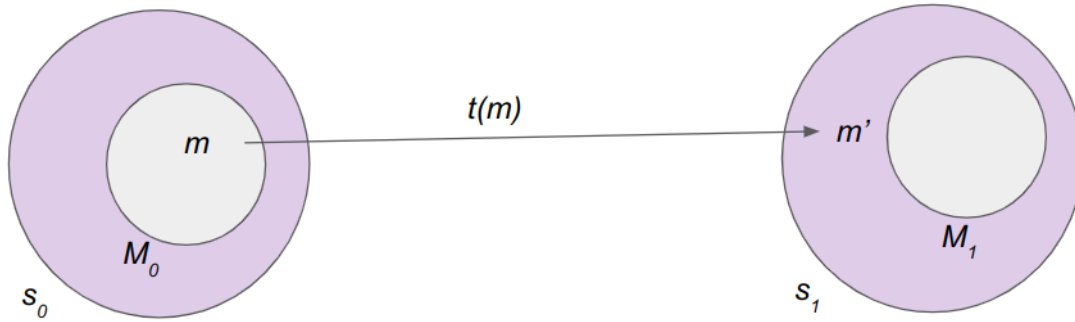


Fig. 2.5: Diagrama de Venn ilustrando el problema a resolver, ie, encontrar una función $t : s_0 \rightarrow s_1$ tal que $t(m) = m'$.

Además, en particular, nos interesa resolver el problema para el dataset D_{Kern} . Para ello, lo que buscaremos es un modelo que represente a t y lo evaluaremos. En el próximo capítulo presentaremos las métricas formales de evaluación del modelo entrenado.

3. MÉTRICAS DE EVALUACIÓN

En esta tesis nos proponemos entrenar un modelo que tome un fragmento musical y lo convierta en uno nuevo que mantenga la identidad del original pero que tenga atributos de un estilo objetivo. Como se mencionó anteriormente, trabajaremos sobre el *dataset* que denominamos D_{Kern} (*dataset* de piezas bajadas de *Kernscores*) que contiene 4 estilos distintos: corales de Bach, canciones de Frescobaldi, sonatas para piano de Mozart y ragtimes. En la bibliografía, los modelos de composición automática suelen evaluarse los resultados mediante encuestas donde se presentan audios y los encuestados evalúan subjetivamente los resultados. Como parte de este trabajo proponemos métricas objetivas para evaluar la calidad de un modelo de transferencia de estilo. En particular, hay 3 ejes que mirar:

- la pertenencia al estilo, es decir, que el fragmento generado suene como el nuevo estilo;
- la musicalidad, es decir, si el fragmento generado suene musical;
- la similitud con el fragmento original, pues no queremos que el fragmento generado sea algo completamente distinto.

A continuación se detallarán las métricas propuestas para cada caso. Para ello, en cada caso proponemos un formalismo para evaluar la característica en particular y luego evaluamos su precisión en el *dataset* D_{Kern} .

3.1. Estilo

Para determinar la pertenencia a un estilo volvemos a remitirnos a su distribución característica basada en Rodríguez Zivic et al. [30]. En la sección 2.2 definimos cómo categorizar un estilo a partir de dos distribuciones de probabilidad que calculamos a partir de un conjunto de fragmentos musicales. Para ello, consideramos las distribuciones de los bigramas de intervalos melódicos $\Sigma^i(M)$ y las distribuciones de los bigramas de patrones rítmicos de 4 unidades de tiempo $\Sigma^r(M)$ y las calculamos sobre los fragmentos de cada estilo, a fin de tener dos distribuciones por estilo. En particular, si podemos calcular estas distribuciones para un conjunto de fragmentos, podemos hacerlo también para un fragmento m solo (considerando a $M = \{m\}$). A partir de estos elementos, podemos medir la distancia que hay entre un fragmento y un estilo. En particular, dada una medida de distancia δ , definimos la diferencia entre fragmento y estilo como la suma de las distancias entre las matrices de intervalos y de ritmos. Es decir, dado un fragmento m , una matriz $\Sigma^r(M)$ de distribución de ritmos asociado a un conjunto M y una matriz $\Sigma^i(M)$ de distribución de intervalos asociado a un conjunto M , definimos la diferencia $\Delta(m, M)$ entre m y M como

$$\Delta(m, M) = \delta(\Sigma^r(m), \Sigma^r(M)) + \delta(\Sigma^i(m), \Sigma^i(M)) \quad (3.1)$$

En particular, tomamos δ como el *transporte óptimo*. El transporte óptimo es una medida que surge de minimizar el costo de convertir una distribución μ a una ν . Esta distancia se suele describir informalmente de la siguiente manera. Supongamos que un obrero

tiene una pala y debe mover una montaña de arena con una forma dada para convertirla en otra forma. Para ello, buscará llevar la arena minimizando el esfuerzo; que lo cuantificamos como la distancia que recorre cargando la arena. En esta analogía, la montaña de arena inicial a la que se quiere llegar son las distribuciones de probabilidad μ y ν . De este modo, el problema consiste en transportar una distribución conocida (la montaña de arena inicial) a otra también conocida minimizando un costo (la distancia entre puntos). En particular, podemos pensar que la masa de un punto x se moverá a otro punto y en las cantidades dadas por una asignación $\pi(x, y)$ y lo que se quiere minimizar es el costo de su transporte $c(x, y)$. Entonces, basándonos en [8] (de donde basamos también la definición informal), definimos el problema de calcular la distancia entre las distribuciones de estilo como buscar el mínimo costo de transportar una distribución μ a una ν . Simbólicamente,

$$\min_{\pi \in \Pi_{\mu, \nu}} \int c(x, y) d\pi(x, y) \quad (3.2)$$

donde Π es el conjunto de asignaciones que convierten a μ en ν . En particular, usaremos la versión discreta de esta ecuación provista por la biblioteca *POT* de Python [10] basada en [26].

3.1.1. Validación de métrica en D_{Kern}

Para validar la métrica, calculamos para cada fragmento de cada estilo cuál es el estilo más cercano según esta medida. Para ello, separamos el dataset en un 80 % (balanceado por estilos) de fragmentos para calcular la distribución de cada estilo y utilizamos el 20 % restante (dataset de test) para mirar el estilo más cercano.

En la Figura 3.1 se observa el resultado de esta medición. Cada subgráfico de barras representa a los fragmentos de cada estilo (del dataset de test) y cada barra la cantidad de fragmentos que tienen al estilo de la barra como el más cercano. Así, por ejemplo, la Figura 2.3c muestra que hay más de 500 fragmentos de estilo Mozart cuyo estilo más cercano es Mozart, pero unos 300 de dicho estilo están más cerca de ragtime. Lo que se busca es que para cada estilo s , la mayoría de los fragmentos de estilo s tengan como estilo más cercano a s .

De este modo comprobamos que la gran mayoría de los fragmentos tienen distribuciones de intervalos y de ritmos que se asemejan más a su propio estilo que al de los otros del dataset.

3.1.2. Evaluación de estilo

Nos preguntamos ahora ¿cómo determinamos que el nuevo fragmento pertenece o no al estilo al que se quiso transformar? Simplemente comparamos las distancias al nuevo estilo antes y después de la transformación. En otras palabras, lo que se busca es que la distancia δ del fragmento transformado m' al nuevo estilo s' sea menor que la del fragmento original m a dicho estilo (s'), es decir:

$$\delta(m', s') < \delta(m, s') \iff \delta(m', s') - \delta(m, s') < 0 \quad (3.3)$$

A partir de esto, la evaluación general consistirá en calcular para cada par estilo de origen - estilo objetivo, el porcentaje de fragmentos transformados que cumplen con esta ecuación.

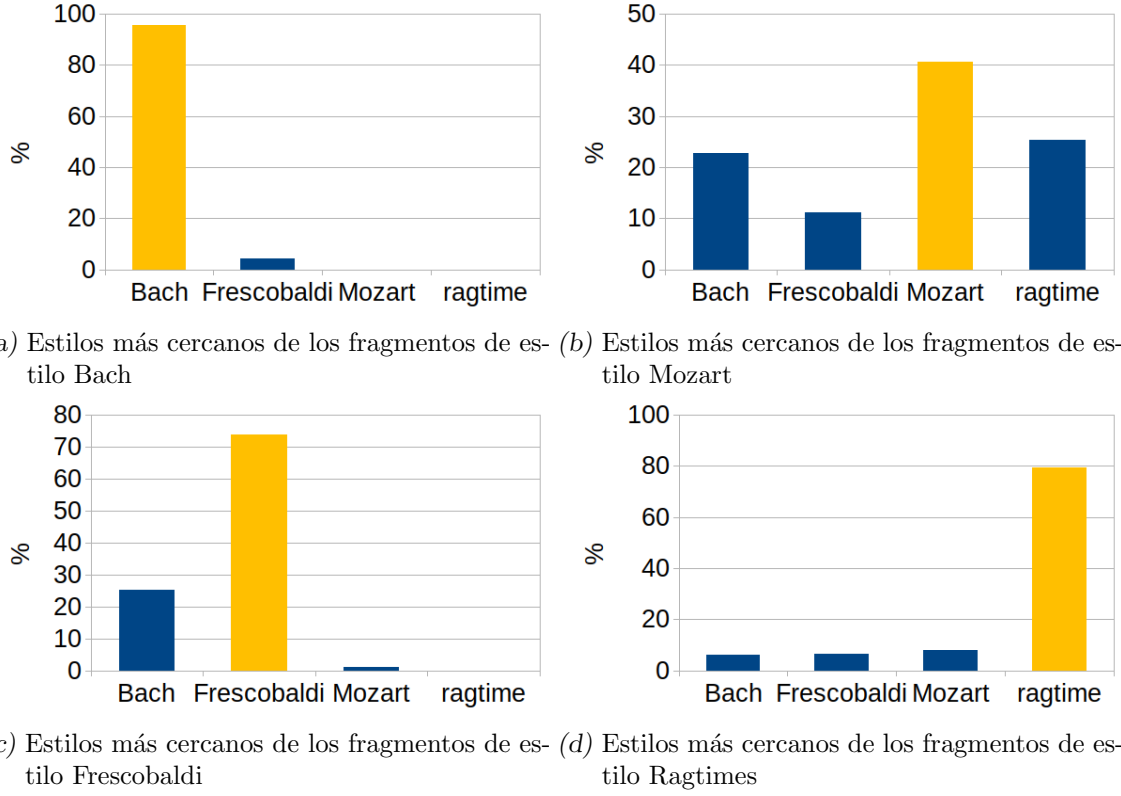


Fig. 3.1: Para cada estilo, cantidad de fragmentos cuyo estilo más cercano es Bach, Mozart, Frescobaldi o ragtime.

3.2. Musicalidad

No hay consenso académico acerca del concepto de *musicalidad* excepto que es una característica propia del ser humano [14]. Definir si una secuencia de notas suena como música es un problema en sí mismo. Para el presente trabajo se trabajará con una definición *ad hoc* asumiendo que hay reglas implícitas del uso de saltos melódicos y rítmicos que hacen que una secuencia de notas suene musical. Proponemos estimar estas reglas implícitas de musicalidad a partir de los datos que tenemos. Para ello, nos basamos en el mismo principio que para estilos pero considerando un “estilo musical”, es decir, consideramos a un fragmento como *musical* si pertenece a dicho estilo *musical*.

3.2.1. Formalización

Para definir el *estilo musical*, se realiza un muestreo balanceado de los 4 estilos de forma tal que haya la misma representación de cada estilo en la definición de musicalidad. Luego, análogamente a la definición de pertenencia a un estilo que se detalló en la sección anterior, se calcula la diferencia de la ecuación 3.4 con el transporte óptimo como δ . Es decir, dado nuestro *dataset* D_{Kern} , tomamos los subconjuntos de fragmentos M_{Bach} , M_{Mozart} , $M_{Frescobaldi}$ y $M_{ragtime}$ de estilos Bach, Mozart, Frescobaldi y ragtime respectivamente, con $\#M_{Bach} = \#M_{Mozart} = \#M_{Frescobaldi} = \#M_{ragtime}$. A continuación definimos $M^{mus} = M_{Bach} \cup M_{Mozart} \cup M_{Frescobaldi} \cup M_{ragtime}$ como el subconjunto que define a nuestro “estilo

musical”. Finalmente calculamos la diferencia de un fragmento m a M^{mus} como:

$$\Delta(m, M^{mus}) = \delta(\Sigma^r(m), \Sigma^r(M^{mus})) + \delta(\Sigma^i(m), \Sigma^i(M^{mus})) \quad (3.4)$$

donde δ es el transporte óptimo y Σ^r y Σ^i son las matrices que describen la distribución de bigramas de patrones rítmicos y la distribución de bigramas de intervalos respectivamente.

3.2.2. Validación de musicalidad en D_{Kern}

Para validar esta métrica buscamos ver si puede capturar alguna noción del concepto de *musicalidad*. Para ello, consideramos que la musicalidad es una característica que depende de la organización de alturas y duraciones de las notas del fragmento musical. Por lo tanto, si rompiéramos esta organización (por ejemplo, reordenando de forma aleatoria las notas), nuestra medida de musicalidad debería verse afectada.

Para validarlo con nuestros datos, nuevamente se separó (balanceado por estilos) el 80 % del *dataset* (que llamamos *train*) y se armó un nuevo conjunto de fragmentos con permutaciones de las notas de los fragmentos originales. En total, se crearon 5 fragmentos nuevos por cada fragmento original perteneciente al 80 %. Finalmente, se calculó la diferencia de cada uno de los fragmentos (del 80 %, del 20 % restante, que llamamos *test* y de las permutaciones) con el *estilo musical*. En particular, lo que esperamos es que las permutaciones tengan valores de musicalidad menores que el resto de los ejemplos, es decir, mayor distancia a la distribución de musicalidad.

En la Figura 3.2 se muestran las distribuciones de diferencias con el *estilo musical* para cada uno de estos tres conjuntos de datos. Dado que el 0 se encuentra a la izquierda del gráfico y las distancias son positivas, lo que interesaría notar es que las distribuciones de las permutaciones se encuentren más a la derecha (más lejos del estilo) que las distribuciones de los fragmentos originales y de *test*. Sin embargo las tres parecen ser similares, lo cual indica que esta no es una métrica que pueda distinguir entre permutaciones aleatorias de las notas y las originales.

Surge entonces la idea de probar con una medida de distancia δ distinta. Proponemos una Δ que evalúa la probabilidad de que un fragmento musical m se produzca a partir de las distribuciones $\Sigma^i(M^{mus})$ y $\Sigma^r(M^{mus})$. En otras palabras, la Δ que usaremos será proporcional a la probabilidad de muestreo $P(m|\Sigma^{i/r}(M^{Mus}))$.

Lo que queremos es calcular la probabilidad conjunta de un fragmento m con nuestra distribución de musicalidad. Esto es:

$$\Delta(m, M^{Mus}) \propto p(m, M^{mus}) = p(m|M^{mus}) \cdot p(M^{mus}) \quad (3.5)$$

Siendo $p(M^{Mus})$ constante, podemos simplificar a:

$$\Delta(m, M^{Mus}) \propto p(m|M^{mus}) \quad (3.6)$$

Para simplificar vamos a considerar a la melodía independiente del ritmo. De esta forma, tenemos:

$$p(m|M^{mus}) = p^i(m|M^{mus}) \cdot p^r(m|M^{mus}) \quad (3.7)$$

con p^i la probabilidad de la secuencia de intervalos de m y p^r la probabilidad de los patrones rítmicos de m .

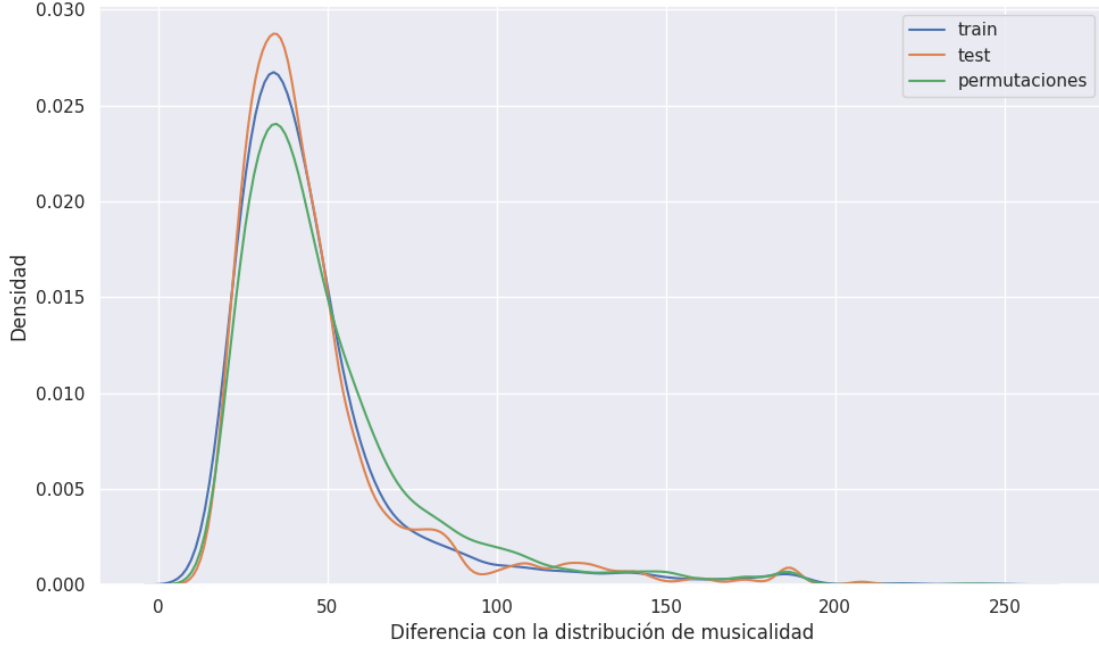


Fig. 3.2: Distribuciones de diferencias (en transporte óptimo) de 3 conjuntos de fragmentos con el *estilo musical*. En azul, la distribución del 80 % de los fragmentos de D_{Kern} (balanceado en estilos), en verde la distribución de las permutaciones de estos fragmentos y en anaranjado la distribución del 20 % restante de D_{Kern} .

Para estimar la probabilidad $p^i(m|M^{Mus})$, podemos considerar a m como una secuencia de intervalos $m_{1..n}^i$. Además, por nuestra categorización de estilo y musicalidad, tenemos que un intervalo m_k^i solo depende del intervalo anterior m_{k-1}^i . De esta forma, podemos definir:

$$p^i(m|M^{Mus}) \propto \prod_{i=2}^n \Sigma^i(M^{mus})_{m_{k-1}^i, m_k^i} \quad (3.8)$$

$$\propto \sum_{i=2}^n \log(\Sigma^i(M^{mus})_{m_{k-1}^i, m_k^i}) \quad (3.9)$$

donde además consideramos que el primer intervalo es equiprobable.

Podemos redefinir δ para melodía, considerando las matrices de estilo Σ^i como:

$$\delta(\Sigma^i(m), \Sigma^i(M^{mus})) = \sum_{i,j} \log(\Sigma_{ij}^i(M^{mus})) \Sigma_{ij}^i(m) \quad (3.10)$$

Esto se debe a que $\Sigma^i(M^{mus})$ nos da la probabilidad de cada bigrama de intervalos y $\Sigma^i(m)$ es el conteo de cada bigrama en m (dividido por un valor que es constante para m y sus permutaciones).

Análogamente, podemos considerar al ritmo de un fragmento m como la secuencia de patrones rítmicos $m_{1..n}^r$. Del mismo modo podemos definir:

$$p^r(m|M^{Mus}) \propto \sum_{i=2}^n \log(\Sigma^r(M^{mus})_{m_{k-1}^r, m_k^r}) \quad (3.11)$$

De forma análoga podemos redefinir δ para ritmos a partir de Σ^r , con las mismas consideraciones que en 3.10:

$$\delta(\Sigma^r(m), \Sigma^r(M^{mus})) = \sum_{i,j} \log(\Sigma_{ij}^r(M^{mus})) \Sigma_{ij}^r(m) \quad (3.12)$$

Ahora sí definimos Δ a partir de la ecuación 3.1, por lo tanto queda:

$$\Delta(m, M^{mus}) = \sum_{i,j} \log(\Sigma_{ij}^i(M^{mus})) \Sigma_{ij}^i(m) + \sum_{i,j} \log(\Sigma_{ij}^r(M^{mus})) \Sigma_{ij}^r(m) \quad (3.13)$$

En particular, dado que la matriz M^{mus} se encuentra normalizada, sus valores serán menores o iguales a 1, por lo que al aplicar el logaritmo quedarán valores no positivos. De este modo, no estamos hablando en rigor que δ sea una distancia sino una diferencia.

En la Figura 3.3 se muestran las distribuciones de diferencias con el *estilo musical* para cada uno de estos tres conjuntos de datos. Aquí, si la diferencia entre los fragmentos y la distribución de musicalidad es cercana a 0 significaría que los fragmentos son *musicales*. En cambio, si está muy lejos del 0, significaría lo contrario, es decir, que los fragmentos no se asemejan a la distribución de musicalidad (o, mejor dicho, es poco probable que hayan sido generados en base a dicha distribución). Dado que el 0 se encuentra ahora a la derecha del gráfico, lo que interesa es ver que las distribuciones de las permutaciones están más a la izquierda que las distribuciones de los fragmentos originales y de *test* y que estas últimas coincidan pero no así la distribución de las permutaciones.

Para verificar esto último, aplicamos el test de Kolmogorov-Smirnov¹ [5] contrastando la distribución de *test* y la de las permutaciones contra la de *train*. El resultado obtenido fue que ambas distribuciones presentan diferencias significativas de *train* ($p < 0,05$). Sin embargo, la distribución de las permutaciones dio valores más extremos: el estadístico D fue aproximadamente 0,28 y el p -valor fue del orden de 10^{-172} . En contraposición, para *test* se obtuvo $D = 0,03$ y p -valor $\approx 10^{-3}$. Verificándose esto, consideramos que esta diferencia sirve para identificar fragmentos menos musicales de fragmentos más musicales.

Para medir la musicalidad de un fragmento musical generado m' proponemos calcular el porcentaje de permutaciones del fragmento original m que están más lejos de M^{mus} que m' , por lo que el caso ideal es que m' tenga el 100 % de permutaciones más lejos de M^{mus} . En particular, haremos un muestreo de 5 permutaciones.

3.3. Similitud con el original

La última pregunta que queremos responder es si el fragmento generado mantiene características similares con el fragmento original. Para ello vamos a usar una métrica de similitud. La idea será generar un ranking de similitud entre el fragmento original y el conjunto compuesto por el fragmento transformado y todos los fragmentos del estilo original (sin el fragmento original). Consideramos entonces que el transformado conservó características del fragmento original si aparece en los primeros puestos del ranking. Es

¹ Usando la biblioteca de Python Scipy.

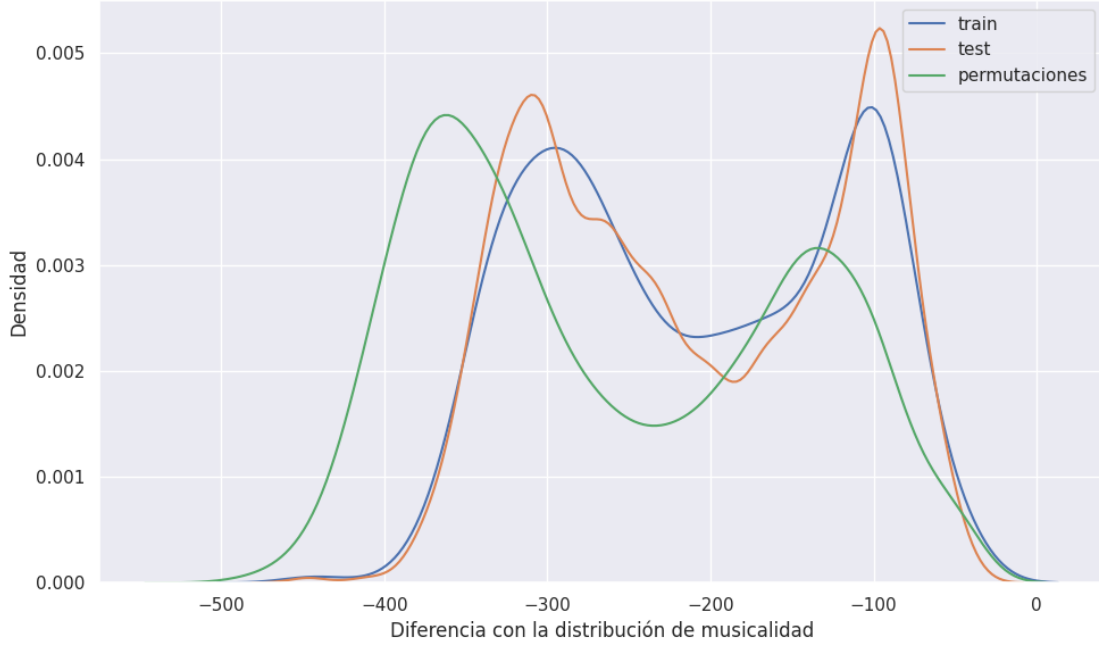


Fig. 3.3: Distribuciones de diferencias (en probabilidad) de 3 conjuntos de fragmentos al *estilo musical*. En azul, la distribución del 80 % de los fragmentos de D_{Kern} (balanceado en estilos), en verde la distribución de las permutaciones de estos fragmentos y en anaranjado la distribución del 20 % restante de D_{Kern} .

decir, el fragmento transformado se parece al original más que los otros fragmentos del mismo estilo (que incluye otros fragmentos de la pieza de donde obtuvimos m). Además, como la cantidad de fragmentos que entran en el ranking dependerá del subconjunto que define al estilo del fragmento original, normalizamos la posición por el cardinal del subconjunto. De este modo, tendremos un valor en el intervalo $[0, 1]$. Ahora bien, este valor será más grande a medida que crezca en la posición del ranking. Para que un valor más grande indique mayor similitud, tomaremos 1 menos esta posición normalizada. Así, para un fragmento original m , su transformado m' y el conjunto M de fragmentos del estilo al que pertenece m definimos:

$$similitud_{\sigma}(m', m, M) = 1 - \frac{Rank(\sigma(m, m'), \{\sigma(m, x) | x \in M \setminus \{m\}\})}{\#M} \quad (3.14)$$

con σ como medida de similitud entre dos fragmentos y $Rank(x, A)$ la posición de x en el ranking generado en $A \cup \{x\}$.

Como medida σ de similitud podríamos usar cualquier métrica de similitud entre fragmentos (puesto que los fragmentos son representados como matrices, podemos usar una distancia matricial). En nuestro caso usaremos dos comparaciones naïves. La primera es comparar tiempo a tiempo si las notas coinciden, contando cuántos instantes tienen la misma nota. A este método lo llamamos *diferencia*. Esta métrica no refleja similitud si, por ejemplo, entre m y m' las notas son las mismas excepto por un semitono de diferencia (en otras palabras, es la misma melodía pero desplazada un semitono) ya que daría como resultado 0 coincidencias cuando en rigor m y m' son muy similares. Así surge la segun-

da métrica que disminuiría este problema: contar para cada instante de tiempo cuánto difieren en altura las notas entre un fragmento del otro. A esta métrica la llamamos de *distancia*.

Estas definiciones hacen que resulte trivial su validación en D_{Kern} e incluso en cualquier *dataset*, ya que un fragmento sin transformar tendrá siempre el mejor valor posible, obteniendo indefectiblemente el primer lugar en el ranking. En otras palabras, $\forall m : M, similitud_{\sigma}(m, m, M) = 1$ con σ definido de alguna de estas dos maneras.

4. PROPUESTA DE ALGORITMO GENERATIVO

Recordemos que el problema que queremos resolver es que dado un fragmento musical m , queremos encontrar una t tal que $t(m) = m'$ sea el fragmento m pero en otro estilo musical; en otras palabras, queremos reconstruir m en otro estilo musical específico. Por otro lado, disponemos de dos *datasets*: D_{Kern} con fragmentos de estilos Bach, Frescobaldi, Mozart y ragtime, y D_{LMD} , un *dataset* más grande con música de diversos estilos. Además, para un fragmento transformado, disponemos de métricas para medir su musicalidad, pertenencia a un estilo y similitud con el fragmento original que usaremos para medir la performance de la transformación. Por lo tanto, si ya podemos caracterizar un conjunto de datos en base a las métricas propuestas, lo que se propondrá ahora es un modelo de aprendizaje automático que permite generar nueva música a partir de información de la ya existente.

4.1. Arquitectura

Consideremos el concepto de Variational Autoencoder (VAE) presentado por primera vez por [18]. Un Autoencoder es una red neuronal profunda con una serie de capas que se encargan de codificar una entrada compleja (imágenes, audio, etc.) llevándola a un espacio vectorial más reducido llamado *espacio latente*. Este conjunto de capas se llama *encoder*. Luego, otra serie de capas (llamadas *decoder*) se encargan de traducir desde ese espacio latente al espacio original buscando reconstruir el mismo objeto que se *encodeó* originalmente. En la Figura 4.1 se muestra un esquema general de este tipo de arquitecturas.

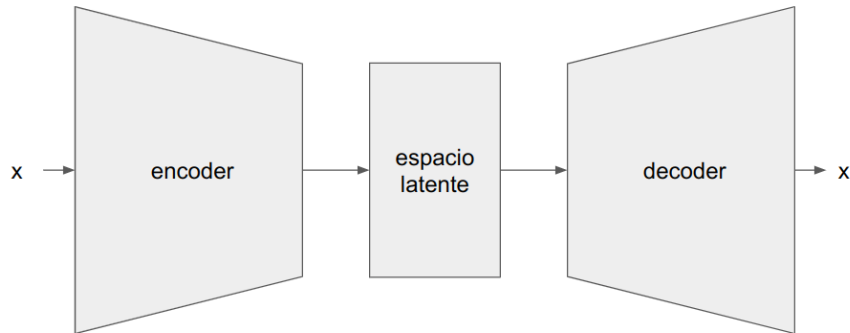


Fig. 4.1: Arquitectura general de un Autoencoder.

Esta reducción de dimensionalidad es posible hacerla gracias a que los atributos de cada *input* no son del todo independientes. Además, gracias a la capacidad de decodificar el espacio latente, es posible aplicar operaciones dentro de dicho espacio de forma que las salidas decodificadas sigan perteneciendo al dominio de interés. En particular, llamando D al dominio del *input* y L al espacio latente, pueden pensarse al *encoder* y al *decoder* como funciones $encode : D \rightarrow L$ y $decode : L \rightarrow D$ respectivamente. De este modo, lo que se busca es que $x = decode(encode(x))$, es decir, que sea la función identidad.

Además, un VAE es un Autoencoder donde se considera un espacio latente probabilístico. De esta forma, el *encoder* codifica la media y la varianza de una distribución del

espacio latente. En particular, es un Autoencoder que agrega un término de regularización Kullback-Leibler (KL) en la distribución del espacio latente [27]. Bajo el supuesto de que la distribución que queremos aproximar es una Normal, esta regularización se utiliza para que dicha Normal se asemeje a una $\mathcal{N}(0, 1)$. De este modo, al finalizar el *encoding* se muestrea la codificación de la entrada en función de esta distribución.

Este tipo de arquitecturas son frecuentemente utilizadas para generar nuevos datos con transformaciones en el espacio latente. Una operación que frecuentemente se realiza cuando muchos *inputs* comparten alguna característica, es calcular el promedio de los vectores *encodeados* de los *inputs* que comparten dicha característica de modo tal de obtener un nuevo vector v llamado *característico* que representa esa característica en común. Este nuevo vector v puede ser luego utilizado para agregar o quitar la característica a otro *input* x mediante la suma o resta, respectivamente, de v a x . En el contexto de imágenes, por ejemplo, se puede calcular el vector característico de “cara con anteojos” para luego, dada una imagen de una cara con anteojos, generar una nueva sin ellos, como se pudo ver en la Figura 1.2 extraído de [15].

A partir de estas consideraciones, se tomó la arquitectura de un VAE presentada por Guo et al. [13]. En ese trabajo, Guo et al. presenta un modelo para agregar o quitar tensión a fragmentos musicales de 4 compases. En particular, el *input* de esta red es el mismo *piano roll* modificado de 89×64 presentado en la sección 2.1. En la Figura 4.2 se muestra el diseño de la arquitectura. Este modelo se encuentra formado en primer lugar por un *encoder* compuesto de 2 capas GRU bidireccionales (la primera de 64×512 y la segunda de 1×512), y luego por dos capas densas separadas de 1×96 con RELU con función de activación que cumplen la función de representar a la media y a la varianza como se explicó anteriormente. De este modo se *encodean* los fragmentos de 4 compases a un espacio latente de 1×96 dimensiones. El *decoder* está compuesto por un Repeat Vector de 64×96 , 2 GRUs de 64×256 y 6 capas densas con funciones de activación RELU, que representan las salidas. Las primeras cuatro son el *piano roll* de entrada pero dividido en cuatro partes: nota del *track* melodía, ritmo del *track* melodía, nota del *track* bajo y ritmo del *track* bajo. La red también tiene como salida otros dos valores adicionales: *cloud diameter* y *tensile strain*. Estas son dos salidas asociadas a la tensión musical, de particular interés para su problema que era transformar un fragmento musical cambiándole el grado de tensión. El primero captura cuán cerca están las notas dentro del espacio tonal, o lo que es lo mismo, la tensión musical en términos de disonancia. El segundo es la distancia entre el centro tonal de las notas del fragmento y su tonalidad. En nuestro caso, esta información no nos interesa, por lo que decidimos excluirla de la salida de la red, quedándonos solo con las 4 anteriores.

4.2. Propuesta de uso

Con estas consideraciones, presentamos nuestra propuesta que es lograr hacer transferencia de estilos mediante aritmética vectorial en el espacio latente del modelo presentado. Lo que proponemos es calcular en dicho espacio el promedio de los vectores de los fragmentos del estilo s . A este vector promedio lo vamos a considerar como el *vector característico* de s y lo llamamos v_s . Estimando estos vectores, podemos proceder a trasladar un fragmento m de estilo s a otro estilo s' .

Para esto, nos basamos en manipulaciones del espacio latente usadas en el dominio del Procesamiento de Lenguaje Natural. En [23] se toma la representación vectorial en

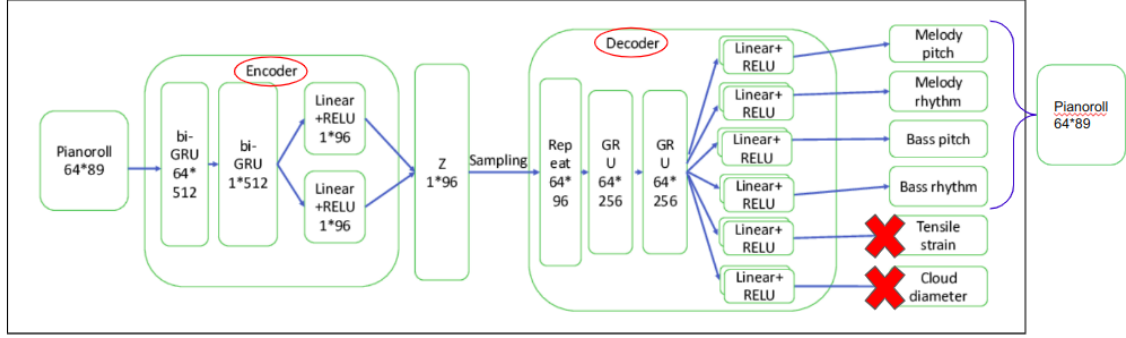


Fig. 4.2: Arquitectura de VAE presentada en Guo et al. [13]. En nuestro caso dejamos de considerar las salidas de *Tensile strain* y *Cloud diameter* quedándonos solo con la funcionalidad de Autoencoder.

el espacio latente de la palabra de la capital de un país a la que llamamos c_0 . Luego, se toman los vectores que representan al país de dicha capital (que llamamos p_0) y el de otro país al que llamamos p_1 . Con estos vectores, la propuesta presentada en dicho trabajo es calcular un nuevo vector c_1 como $c_1 = c_0 - p_0 + p_1$. Este vector resultante resulta tener como vecino más cercano a la capital del segundo país.

A partir de esta propuesta, planteamos dos métodos. Con el primero le sumamos a la codificación del fragmento directamente la *característica* $v_{s'}$ del nuevo estilo s' escalado por un α entre 0 y 1 (ver ecuación 4.1). Con el segundo, además le restamos la *característica* v_s del estilo original s , escalado también por el mismo α , (ver ecuación 4.2). Luego le aplicamos *decode* a este vector resultante y obtendríamos el m' que buscamos. En síntesis, proponemos realizar las siguientes transformaciones:

$$t_{s,s'}(m) = \text{decode}(\text{encode}(m) + \alpha \cdot v_{s'}) = m' \quad (4.1)$$

a la que llamamos *transformación de suma* y

$$t_{s,s'}(m) = \text{decode}(\text{encode}(m) + \alpha \cdot v_{s'} - \alpha \cdot v_s) = m' \quad (4.2)$$

a la que llamamos *transformación de suma y resta*, en ambos casos con $\alpha \in [0, 1]$. En particular, probamos para $\alpha \in \{0, 1, 0, 25, 0, 5, 0, 75, 1\}$ a modo de interpolación entre m y el m' resultante de usar $\alpha = 1$.

4.3. Modelos

Como adelantamos en la introducción, en primer lugar entrenamos la arquitectura como Autoencoder con el *dataset* D_{Kern} , luego con D_{LMD} y finalmente tomando este último modelo y aplicando *fine-tuning* en D_{Kern} . Entrenarla como Autoencoder significa que lo que se busca es que aproxime a la función identidad con el *encoder* parametrizado por ϕ usado para mapear un fragmento al espacio latente z y el *decoder* parametrizado por θ que mapea de z a las 4 salidas $[y_1 \dots y_4]$ que representan al fragmento. En particular, y_1 corresponde a la nota del *track* melodía (con 73 valores posibles), y_2 al ritmo de este *track* (con 2 salidas posibles), y_3 a la nota del *track* bajo (con 12 salidas posibles) e y_4 al ritmo de este *track* (con también 2 salidas posibles). De este modo, definimos la función de pérdida de reconstrucción como la suma de la entropía cruzada en cada una de estas

salidas. Así, la función de pérdida general utilizada fue la *loss* de reconstrucción menos la divergencia K-L de la probabilidad *a priori* $p(z)$ ($\mathcal{N}(0, 1)$) de la distribución que describe al espacio latente y la distribución *a posteriori* $q_\phi(z|x)$ aprendida por el *encoder* [18]. Simbólicamente,

$$\mathcal{L} = L_{rec}(\theta, \phi, x) - \beta D_{KL}(q_\phi(z|x)||p(z)) \quad (4.3)$$

donde β es usado para cambiar el balance de la *loss* de reconstrucción y la *KL-loss*. Respecto a este último parámetro, se lo inicializó en 0 y se lo incrementó en cada *batch* por $5 \cdot 10^{-7}$ hasta llegar a 0,006. Este es el mismo proceso de entrenamiento usado en [13]. Además, el *learning rate* fue 0,0001 y se utilizó *early stopping* con una paciencia de 20 épocas.

En cuanto a los *datasets*, hicimos un *split* estratificado de cada *dataset* en 80 % para entrenamiento, 10 % para test y 10 % para validación. El subconjunto de validación no fue utilizado en esta instancia ya que no se realizó ninguna búsqueda de hiperparámetros. Como en ambos *datasets* los estilos se encuentran desbalanceados, se utilizó *oversampling* para mitigar este problema.

Con estas consideraciones se entrenó el primer modelo. Fue entrenado sobre D_{Kern} durante 95 épocas, finalizando el entrenamiento por *early stoppping*. En la Figura 4.3 se observa el valor de las funciones de pérdida para *train* y *test* en función de la época. Se puede notar cómo va disminuyendo paulatinamente a medida que la red va aprendiendo y cómo se estanca la *loss* de *test* hacia la época 60 alcanzando el mínimo en la 75 con un valor de 4,68.

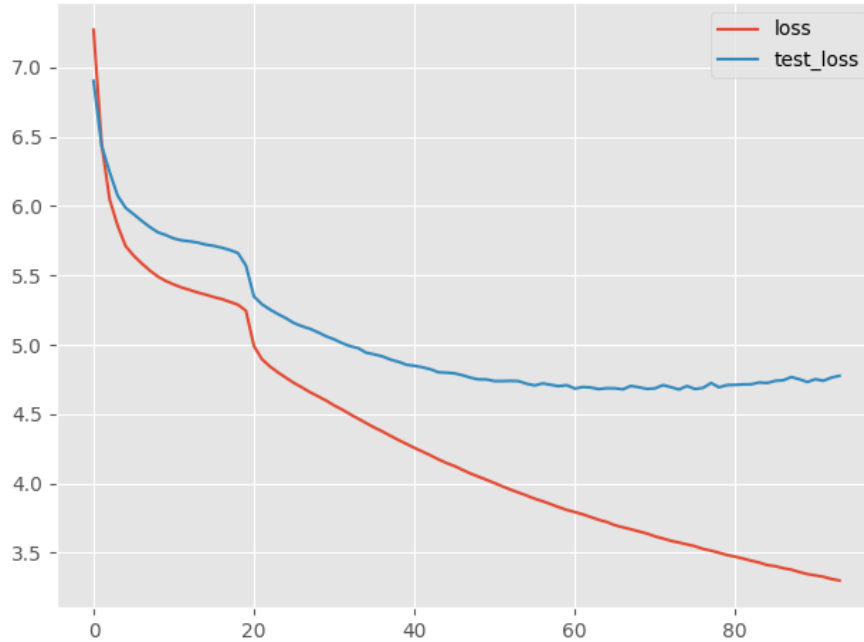


Fig. 4.3: *Loss* en función de la época del modelo entrenado sobre el *dataset* D_{Kern} . En rojo la *loss* para *train* y en celeste para *test*.

Además, en la Figura 4.4 se pueden observar las *accuracies* para cada una de las 4 salidas de la red, para *train* y para *test*. Las asociadas a la melodía de los *tracks* *melodía* y *bajo* son categóricas y las asociadas al ritmo son binarias. Se puede notar así cómo las

categorías son menores que las binarias pero mejoran con el paso de las épocas. Además, todas superan a *random* (que es 0,5 para las binarias y $\frac{1}{12}$ o $\frac{1}{73}$ para las categóricas) en validación. Sin embargo, hay una marcada diferencia entre las *accuracies* de *test* comparado con las de *train*.

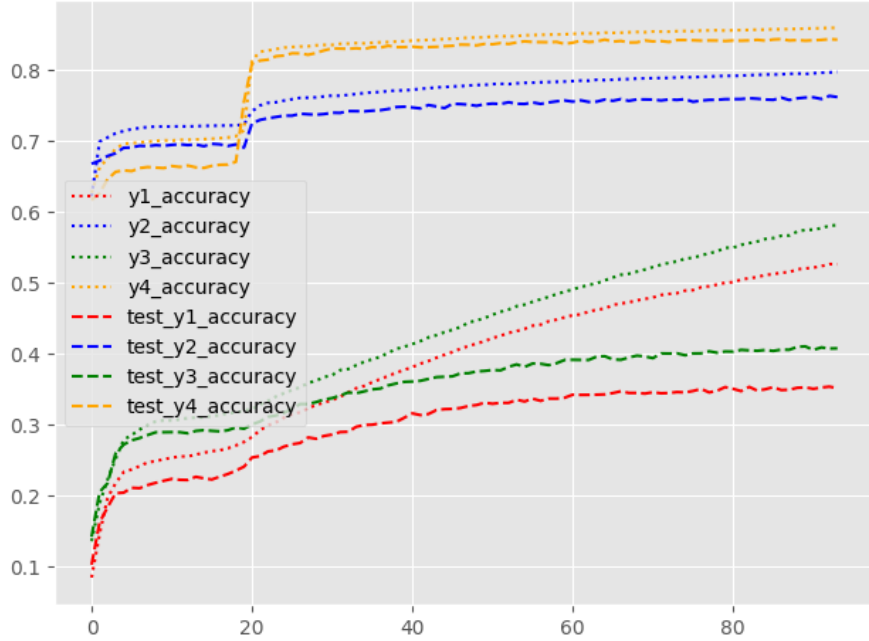


Fig. 4.4: *Accuracies* para cada una de las salidas en función de la época del modelo entrenado sobre el dataset D_{Kern} .

Es así que decidimos entrenar un nuevo modelo sobre un *dataset* más grande que el anterior buscando que pueda aprender los conceptos más generalizables de musicalidad. Para ello escogimos D_{LMD} , 40 veces más grande que el anterior y de fácil acceso como se explicó en la sección 2.3. El entrenamiento duró 484 épocas, finalizando el entrenamiento por *early stopping*. En la Figura 4.5 se observa nuevamente el valor de las funciones de pérdida para *train* y *test* en función de la época para los conjuntos de *train* y de *test*. El valor mínimo de la función de pérdida fue 0,32. En la Figura 4.6 se observan nuevamente las *accuracies* para cada una de las salidas. En todos los casos se obtuvieron *accuracies* mayores al 97%.

Finalmente, a este último modelo se lo tomó como base para aplicar *fine-tuning* sobre D_{Kern} . En este proceso no fue congelada ninguna capa. Así, el modelo *fine tuneado* fue entrenado durante 31 épocas y alcanzó su mínima *loss* para *test* con un valor de 3,17 como se puede observar en la Figura 4.7. En la Figura 4.8 se grafican las *accuracies* con resultados más bajos que en el modelo anterior pues, para validación, la *accuracy* del contorno melódico del *track* melodía no llega a 0,6. De todos modos, sigue siendo superior a *random* y es mejor que el primer modelo.

Para diferenciar entre los tres modelos, llamaremos *base* al primer modelo entrenado solo en D_{Kern} , *pre fine-tuning* al modelo entrenado en D_{LMD} y *post fine-tuning* al modelo *fine-tuneado* en D_{Kern} . De este modo, en el próximo capítulo vamos a comparar los distintos resultados para los tres modelos para determinar si es necesario o no aplicar

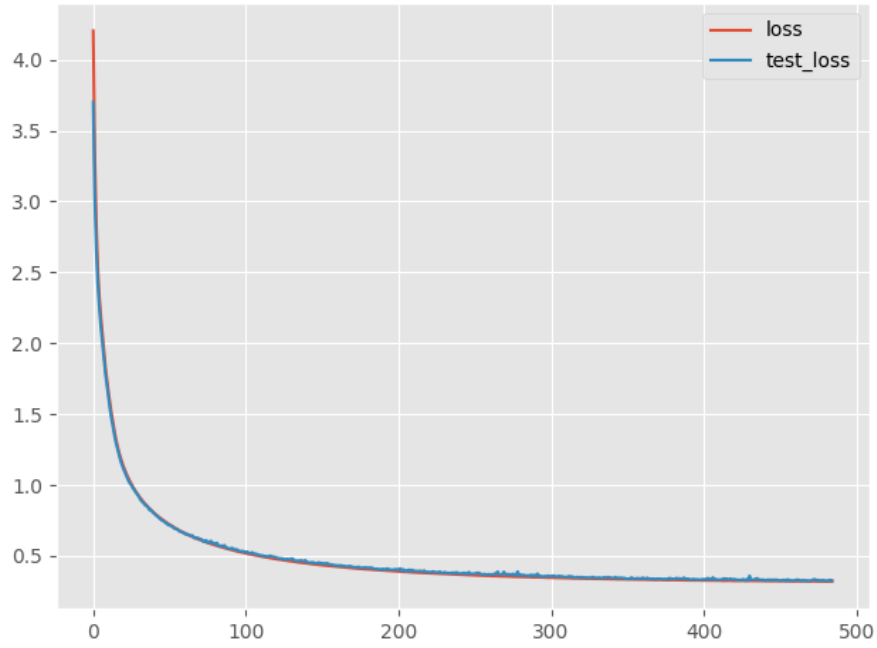


Fig. 4.5: *Loss* en función de la época del modelo entrenado sobre el *dataset* D_{LMD} . En rojo la *loss* para *train* y en celeste para *test*.

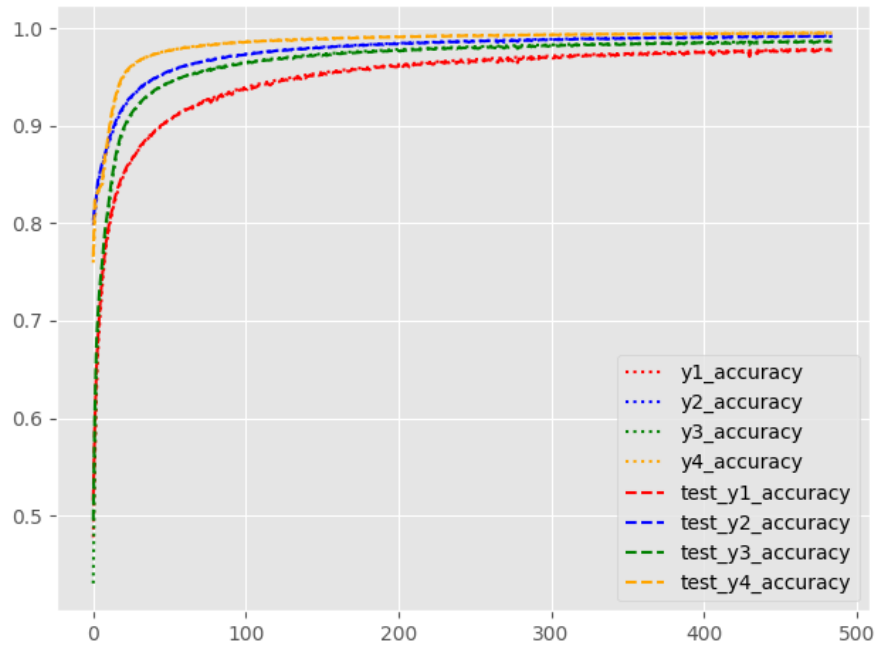


Fig. 4.6: *Accuracies* para cada una de las salidas en función de la época del modelo entrenado sobre el *dataset* D_{LMD} .

fine-tuning para esta tarea con esta arquitectura.

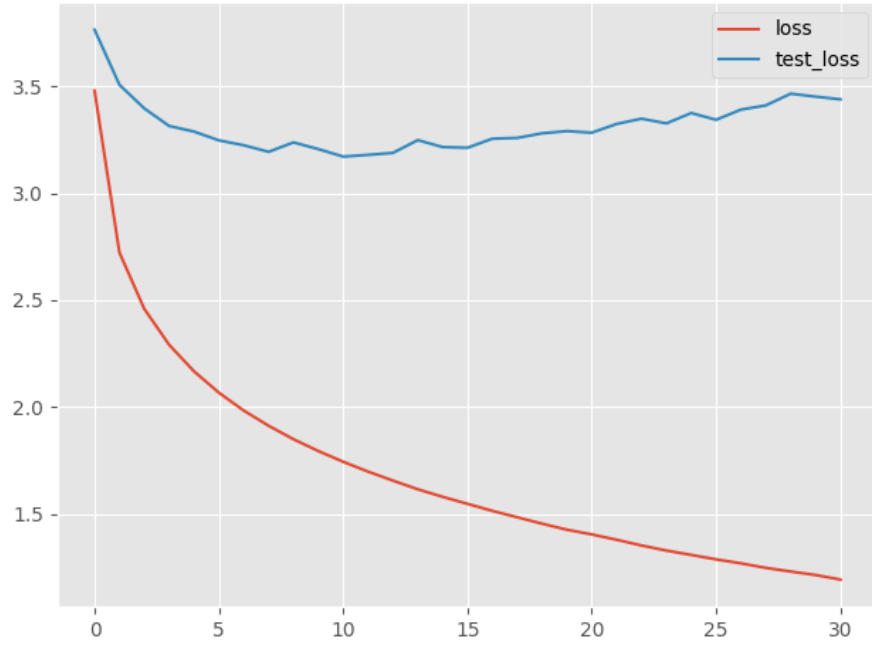


Fig. 4.7: *Loss* en función de la época del modelo entrenado sobre el *dataset* D_{LMD} y *fine-tuneado* sobre D_{Kern} . En rojo la *loss* para *train* y en celeste para *test*.

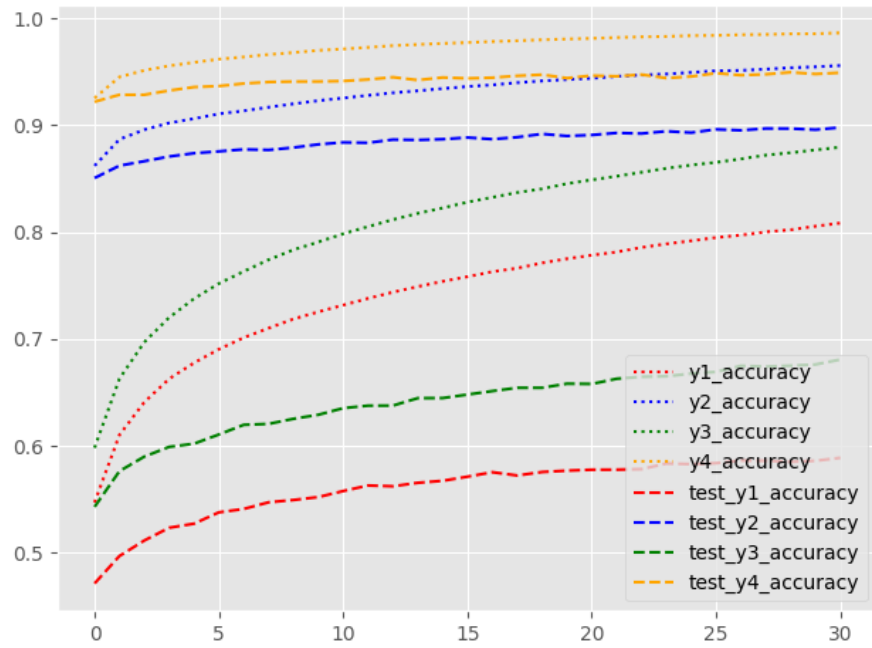


Fig. 4.8: *Accuracies* para cada una de las salidas en función de la época del modelo entrenado sobre el *dataset* D_{LMD} y *fine-tuneado* sobre D_{Kern} .

5. RESULTADOS

Teniendo ya tres modelos entrenados y métricas de evaluación validadas en el *dataset*, estamos en condiciones de evaluar los resultados. Para ello, generamos transformaciones de todos los fragmentos a cada uno de los otros estilos. En particular, realizamos las transformaciones de *suma* y de *suma y resta* con los valores de $\alpha \in \{0,1; 0,25; 0,5; 0,75; 1\}$. De este modo, se generaron $3850 \cdot 3 \cdot 2 = 23100$ fragmentos nuevos (3.850 fragmentos originales, 3 estilos objetivo posibles, 2 métodos de transformación) para cada α , es decir, en total, 115.500 fragmentos. Dada la imposibilidad de escucharlos a todos, se seleccionó aleatoriamente a un fragmento por estilo y se escuchó cada una de las transformaciones a cada estilo y para cada α , a fin de evaluar informalmente la capacidad de los modelos. Es decir, por cada uno de los 4 estilos se escucharon $2 \cdot 3 \cdot 5 = 30$ fragmentos nuevos. Finalmente evaluamos a todos en los 3 puntos considerados, a saber:

- Si el fragmento producido es musical.
- Que se haya cambiado el estilo en sí. Es decir, que suene más parecido al nuevo estilo que antes.
- Que no haya perdido las características del fragmento original (los modelos podrían generar piezas del estilo objetivo sin reflejar el fragmento original).

Considerando nuestras métricas propuestas, intentamos responder las preguntas:

- ¿El nuevo fragmento está más cerca (en términos de probabilidad) del *estilo musical* que permutaciones del fragmento original?
- ¿Está más cerca (usando transporte óptimo) que antes del nuevo estilo?
- ¿Qué posición del ranking de similitud ocupa en relación a todos los fragmentos del estilo?

5.1. Escucha general

Como ya se dijo, en primer lugar se procedió a escuchar los resultados de las transformaciones para una selección aleatoria de fragmentos de cada estilo con los distintos valores de α señalados anteriormente.

Lo primero que se nota en los resultados de ambos modelos es que la fidelidad se va perdiendo a medida que el α va creciendo, es decir, el parecido con el original es mayor cuando α es chico, lo cual tiene sentido porque la transformación es menor. Por otro lado, la musicalidad se conserva con valores intermedios de α , pero puede degradarse si el α es muy grande. En este sentido, es importante la graduación de dicha variable para evitar demasiada corrupción en la musicalidad. Finalmente, en cuanto al acercamiento a estilos, esto puede ser difícil de evaluar auditivamente. En muchos casos, la transformación efectivamente modifica el fragmento y lo mantiene musical, pero es más difícil definir cuánto está tomando del nuevo estilo. En las escuchas realizadas, la transformación fue más evidente hacia los estilos Bach y ragtime cuando α crece. Hipotetizamos que esto se debe a que estos estilos se encuentran en los lugares extremos en complejidad como se observó en la Figura

2.4. Por el contrario, el acercamiento a los estilos Mozart y Frescobaldi parece ser menor. En el siguiente enlace puede encontrarse el ejemplo escuchado para cada tipo de transformación hechas con el modelo *post fine-tuning*: <https://docs.google.com/presentation/d/1W0koeRrcMDi2010510pKz1m6vkz-8DSpd15060S7xR4/edit?usp=sharing>.

En las siguientes secciones, presentaremos las evaluaciones de musicalidad, acercamiento al estilo objetivo y similitud con el fragmento original usando las métricas desarrolladas en el capítulo 3. Para ello, calcularemos los valores para cada par de estilos origen-objetivo, presentando primero una comparación entre los dos métodos de transformación (*suma* vs. *suma y resta*, correspondientes a las ecuaciones 4.1 y 4.2 respectivamente) con el modelo *post fine-tuning* y luego, tomando el segundo de los métodos, comparamos el desempeño con los modelos *base*, *pre* y *post fine-tuning*. En todos los casos mostramos los resultados para los valores de $\alpha \in \{0,1; 0,5; 1\}$.

5.2. Musicalidad

Para evaluar el sistema en su conjunto, para cada fragmento medimos el porcentaje de permutaciones del fragmento original que son menos musicales (que están más lejos del *estilo musical*) que el fragmento transformado. De este modo, un fragmento tendrá una calificación mejor si el porcentaje es alto, es decir, si está más cerca del *estilo musical* que sus permutaciones. Comparamos el promedio de estos porcentajes para cada par estilo original - estilo objetivo.

5.2.1. Resultados

En la Figura 5.1 se observan los resultados de evaluar musicalidad con los métodos de las ecuaciones *suma* y *suma y resta* para el modelo *post fine-tuning*. Los ejes y muestran los distintos estilos de origen s y los ejes x los distintos estilos objetivo s' de cada transformación. Se puede observar de este modo que al sumar $\alpha v_{s'}$ y restar αv_s se pierde más musicalidad que si solo se suma $\alpha v_{s'}$ para los casos en los que se parte de Frescobaldi o de Bach. Esto se condice con pasar de estilos de menor complejidad rítmica y melódica a estilos de mayor complejidad. En el resto de los casos se dan resultados cercanos al óptimo, sin diferencias notables entre métodos excepto las transformaciones entre Mozart y ragtime que son mejores con el método de la *suma y resta*. Además, se aprecia cómo el mayor valor de α implica mayor pérdida de musicalidad. Las mismas conclusiones pueden darse analizando los dos métodos pero con los otros dos modelos.

En la Figura 5.2 se observan los resultados de evaluar musicalidad con el método de *suma y resta*. En 5.2a, 5.2b y 5.2c se grafican los resultados para el modelo *base*, en 5.2d, 5.2e y 5.2f para el modelo preentrenado en D_{LMD} (*pre*) y en 5.2g, 5.2h y 5.2i para el modelo *fine-tuneado* en D_{Kern} (*post*). Se puede apreciar, al igual que antes, cómo se degrada la musicalidad en algunos casos a medida que crece α . Además, la musicalidad mejora a medida que se complejiza el modelo para α chico, mientras que para α grande sucede lo contrario. Por otro lado, usar a Mozart o ragtime como estilos de partida parecen ser bastante sólidos en tanto no se percibe casi degradación de musicalidad a medida que la transformación es mayor.

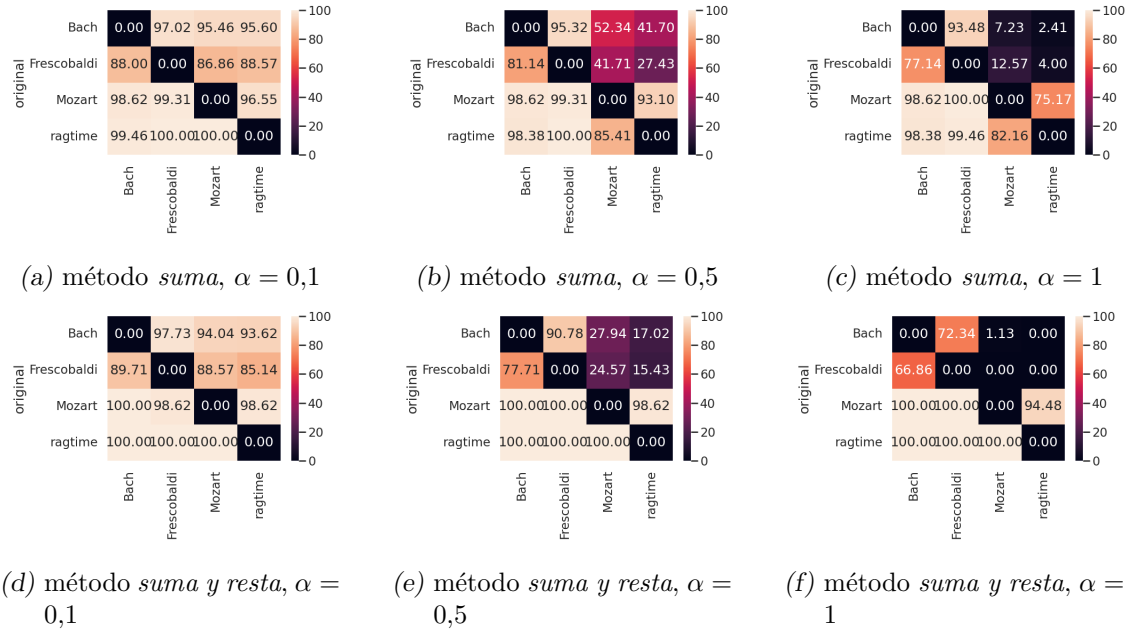


Fig. 5.1: Comparación de resultados de evaluar musicalidad con los métodos de las ecuaciones 4.1 (*suma*, fila superior) y 4.2 (*suma y resta*, fila inferior) para el modelo preentrenado en $D_{LMD}(pre\ fine-tuning)$ y *fine-tuneado* en $D_{Kern}(post\ fine-tuning)$. Los ejes y muestran los distintos s y los ejes x los distintos s' de cada transformación. Los valores corresponden al promedio de porcentajes de permutaciones del fragmento original que son menos musicales que el fragmento transformado.

5.3. Estilo

Para la evaluación de estilo nos interesa ver si los fragmentos transformados se acercan al estilo objetivo. Acercarse significa que la distancia del fragmento transformado al estilo es menor que la del fragmento original, como expresamos en la ecuación 3.3. En particular, lo que mediremos es qué porcentaje de fragmentos cumplen con esta condición para cada tipo de transformación.

Al igual que con musicalidad, comparamos en primer lugar los resultados con los dos métodos de las ecuaciones de *suma* y de *suma y resta* y distintos valores de α para el modelo *post fine-tuning*. Análogas conclusiones pueden extraerse de analizar los dos métodos con los otros dos modelos. Luego, compararemos el rendimiento para los tres modelos.

5.3.1. Resultados

Los gráficos de la Figura 5.3 representan para cada estilo origen (eje y), el porcentaje de fragmentos transformados que se acercaron al nuevo estilo (eje x). Como es esperable, si la transformación es débil (α es chico), el acercamiento será bajo, como se observa en 5.3a y 5.3d. Lo interesante a notar es que a medida que α crece, hay más fragmentos que se acercan al nuevo estilo, que es justamente lo que buscábamos. Pero si crece demasiado también puede degradarse, como el caso de las transformaciones entre ragtime y Mozart para el método de la *suma*, o para todos los casos con el método de la *suma y resta*. Por otro lado, para este método es mejor usar $\alpha = 0,5$ mientras que para *suma* es mejor $\alpha = 1$. Esto sugiere que puede ser necesario parametrizar a la resta independientemente

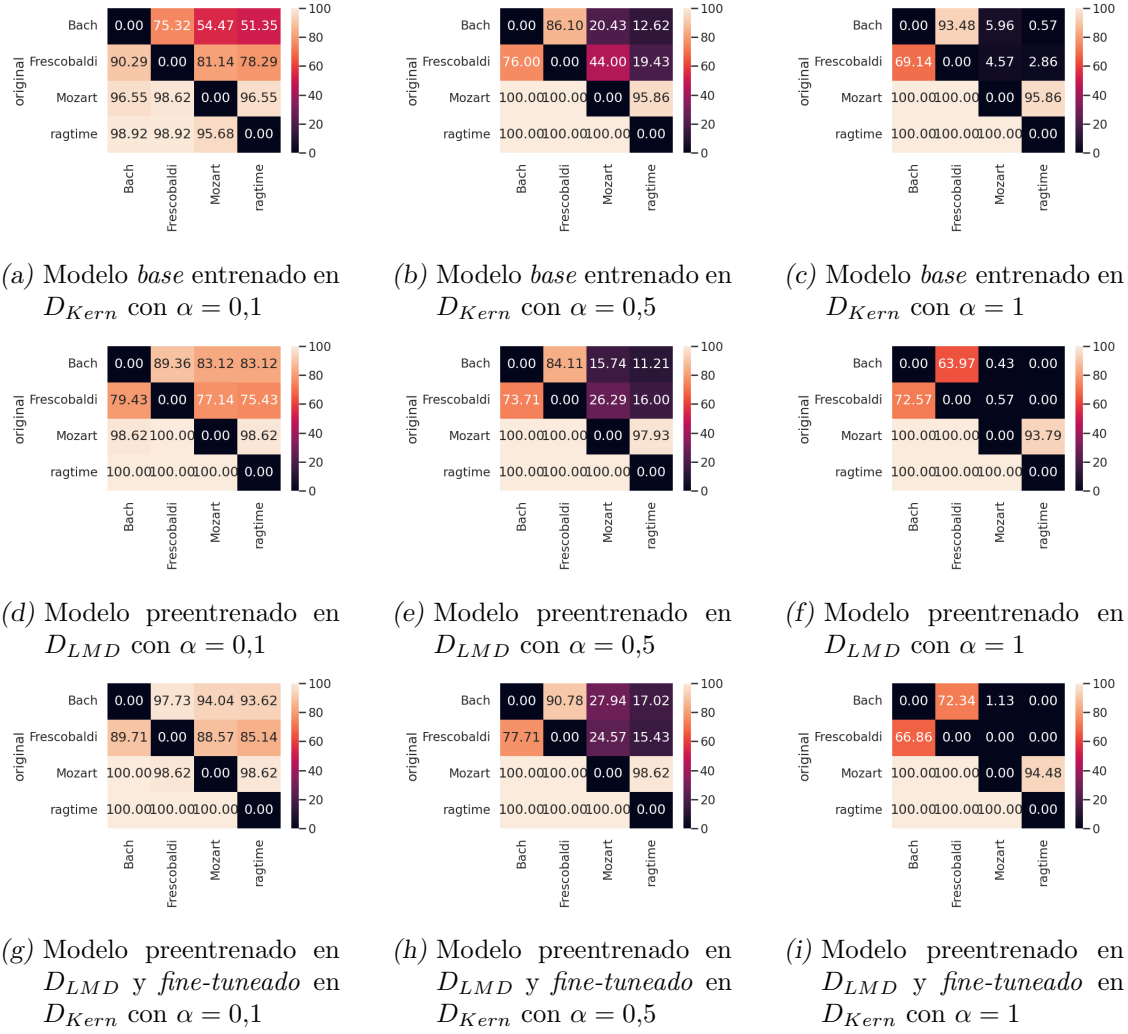


Fig. 5.2: Comparación de resultados de evaluar musicalidad con el método de *suma y resta* para el modelo *base* entrenado en D_{Kern} (fila superior), para el *pre fine-tuning* entrenado en D_{LMD} (fila del medio) y luego para el *fine-tuneado* en D_{Kern} (*post fine-tuning*, fila inferior). Los ejes y muestran los distintos s y los ejes x los distintos s' de cada transformación. Los valores corresponden al promedio de porcentajes de permutaciones del fragmento original que son menos musicales que el fragmento transformado.

de la suma.

En la Figura 5.4 se pueden observar los resultados de la evaluación de estilos para el método de *suma y resta* para el modelo *base* (figuras 5.4a, 5.4b y 5.4c), *pre fine-tuning* (figuras 5.4d, 5.4e y 5.4f) y para el modelo *post fine-tuning* (figuras 5.4g, 5.4h y 5.4i) con distintos valores de α nuevamente. Aquí también se observa que a mayor transformación, mayor es el acercamiento al estilo objetivo. Se observa en primer lugar una clara mejoría de los modelos *pre* y *post* respecto del *base* independientemente del valor de α . Por otro lado, el modelo *pre fine-tuning* tiene similares resultados (e incluso en algunos casos, mejores) que el estilo *post fine-tuning* (salvo para α chico donde *pre* es un poco mejor). Además, en los tres modelos, las transformaciones entre Mozart y ragtime parecen tener las mayores dificultades para acercarse entre sí.

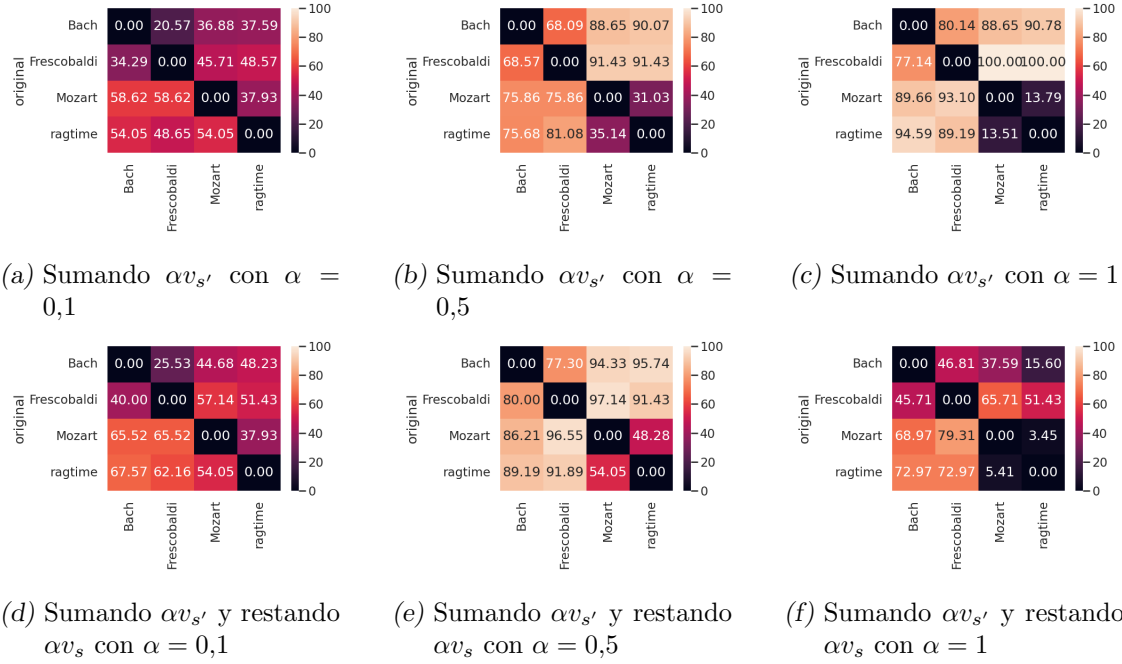


Fig. 5.3: Comparación de resultados de evaluar el acercamiento al nuevo estilo con los métodos de *suma* y de *suma y resta* para el modelo *post fine-tuning*. Los ejes y muestran los distintos s y los ejes x los distintos s' de cada transformación. Los valores corresponden al porcentaje de fragmentos transformados que se acercaron al estilo objetivo.

5.4. Similitud al original

Finalmente, en cuanto a similitud con el fragmento original, analizamos el promedio de aplicar la función de similitud para cada uno de los fragmentos transformados. A continuación presentamos el promedio de estos puntajes para las transformaciones del estilo original s al estilo objetivo s' . Recordemos que dicha función fue definida en 3.14 como 1 menos la posición normalizada en el ranking generado de medir similitud contra el fragmento original entre el fragmento generado y los fragmentos del estilo original. Lo que buscamos es que los valores sean lo más cercanos a 1 como sea posible. En particular, analizamos lo que sucede utilizando las dos funciones de similitud entre fragmentos propuestas: *diferencia* y *distancia*. La primera es comparar tiempo a tiempo si las notas coinciden, contando cuántos instantes tienen la misma nota. La segunda consiste en contar para cada instante de tiempo cuánto difieren en altura las notas entre un fragmento del otro.

Al igual que en los casos anteriores, comparamos los dos métodos de transformación de las ecuaciones *suma* y *suma y resta* y distintos valores de α para el modelo *post fine-tuning* (resultados análogos se obtienen analizando los otros modelos). Hacemos esto para ambas métricas de similitud. Luego analizamos la diferencia entre los tres modelos para el método de *suma y resta*.

5.4.1. Resultados

Graficamos entonces en la Figura 5.5 el promedio de la función de fidelidad que habíamos definido para cada tipo de transformación, en particular, utilizando a la *di-*

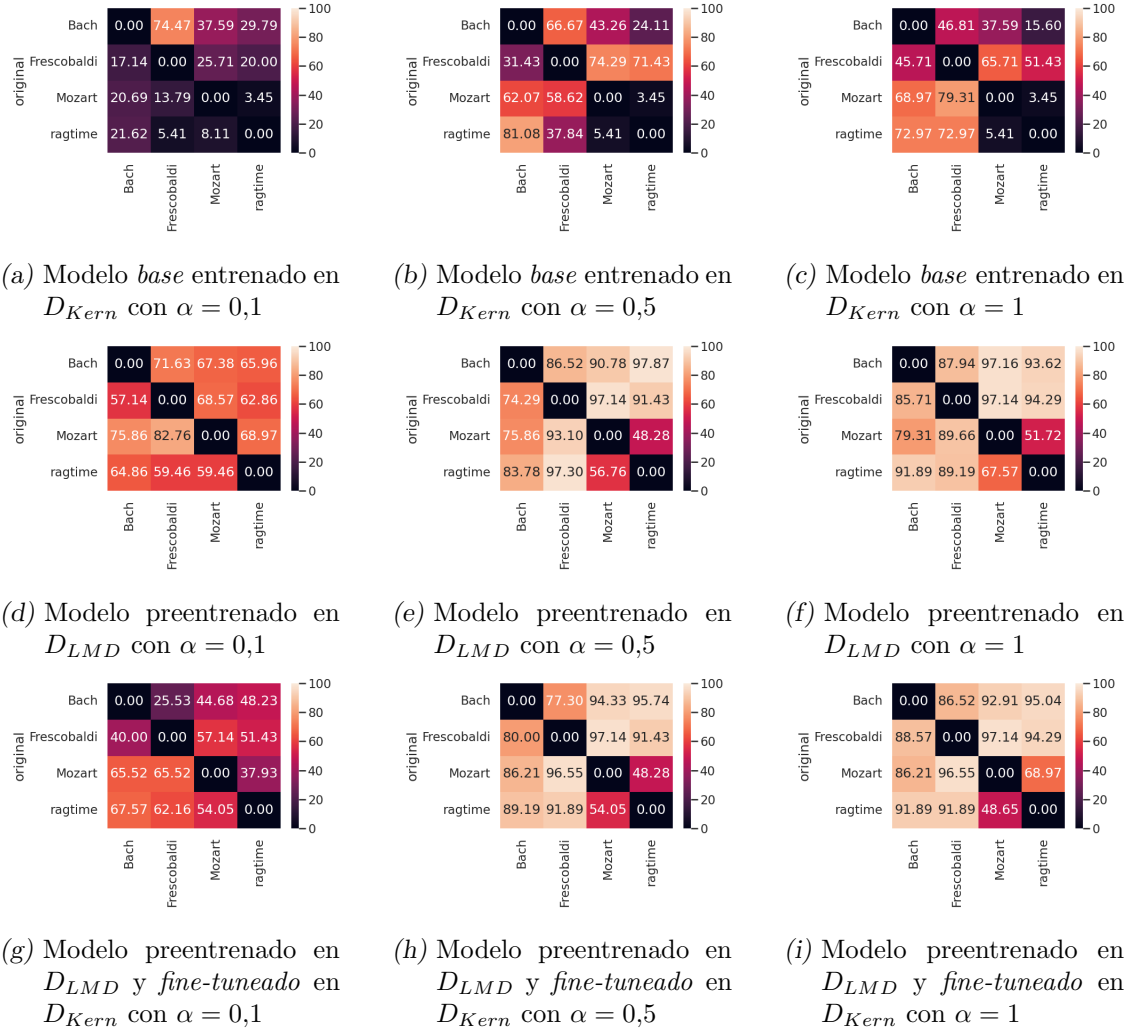


Fig. 5.4: Comparación de resultados de evaluar acercamiento al estilo objetivo con el método de *suma y resta* para el modelo *base* entrenado en D_{Kern} (fila superior), para el *pre fine-tuning* entrenado en D_{LMD} (fila del medio) y luego para el *fine-tuneado* en D_{Kern} (*post fine-tuning*, fila inferior). Los ejes y muestran los distintos s y los ejes x los distintos s' de cada transformación. Los valores corresponden al porcentaje de fragmentos transformados que se acercaron al estilo objetivo.

ferencia como función de similitud. Aquí nuevamente se puede observar cómo se degrada (aunque levemente) la performance a medida que crecen los valores de α . Por otro lado, son similares los resultados entre los dos métodos, siendo levemente peor el de *suma y resta* cuando $\alpha = 1$ y Bach es el estilo de origen u objetivo de la transformación.

Graficamos luego en la Figura 5.6 el promedio de la función de fidelidad para cada tipo de transformación utilizando ahora a la *distancia* como función de similitud. Aquí se observa más fuertemente la caída en la performance a medida que crecen los valores de α . Es notorio en particular cómo disminuye para las transformaciones con Bach como estilo original u objetivo (salvo cuando el original es Frescobaldi) pero solo para el método de *suma y resta* con $\alpha = 1$. Esto también sugiere que puede ser necesario parametrizar a la resta de forma independiente a la suma y muestra que el método de similitud de la

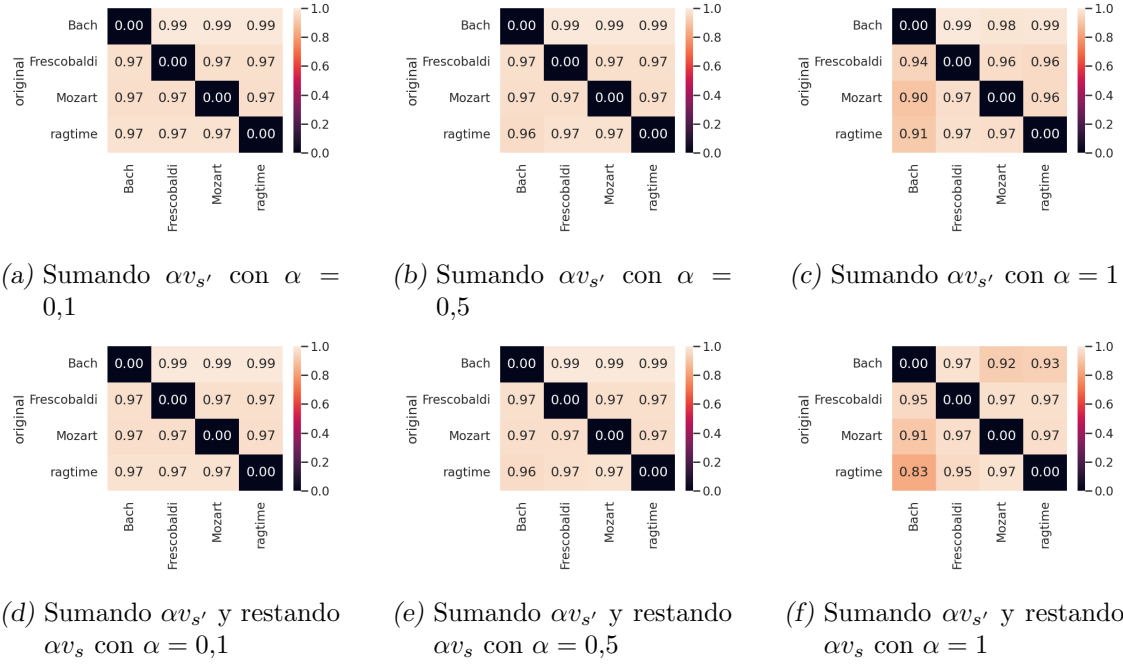


Fig. 5.5: Comparación de resultados de evaluar la función de fidelidad con el método de similitud de la *diferencia* con los métodos de *suma* y *suma y resta* para el modelo *post fine-tuning*. Los ejes y muestran los distintos s y los ejes x los distintos s' de cada transformación. Los valores corresponden a los promedios de evaluar el método de la *diferencia* en los fragmentos transformados de la transferencia correspondiente.

distancia es más sensible que el de la *diferencia* como adelantamos en su definición.

Las Figuras 5.7 y 5.8 representan las comparaciones entre los modelos *base*, *pre* y *post fine-tuning*, en el primer caso usando a la *diferencia* como medida de similitud y en el segundo a la *distancia*. Aquí se observan resultados similares a los anteriores: la pérdida de similitud a medida que α crece y cómo para las transformaciones que contienen a Bach (salvo cuando Frescobaldi es el original) se degradan a medida que α crece. Esto además se observa más cuando se usa la *distancia* como función de similitud. Por otro lado, no se observan diferencias notorias entre los tres modelos, siendo levemente peor el modelo *base*.

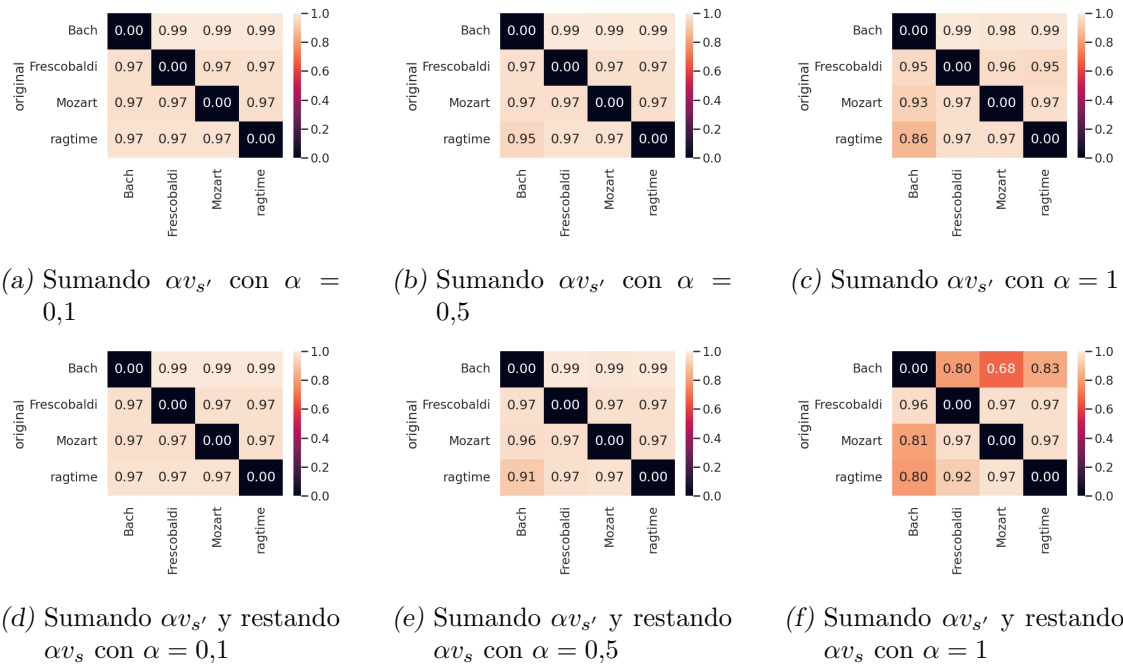


Fig. 5.6: Comparación de resultados de evaluar la función de fidelidad con el método de similitud de la *distancia* con los métodos de *suma* y *suma y resta* para el modelo *post fine-tuning*. Los ejes *y* muestran los distintos *s* y los ejes *x* los distintos *s'* de cada transformación. Los valores corresponden a los promedios de evaluar el método de la *distancia* en los fragmentos transformados de la transferencia correspondiente.

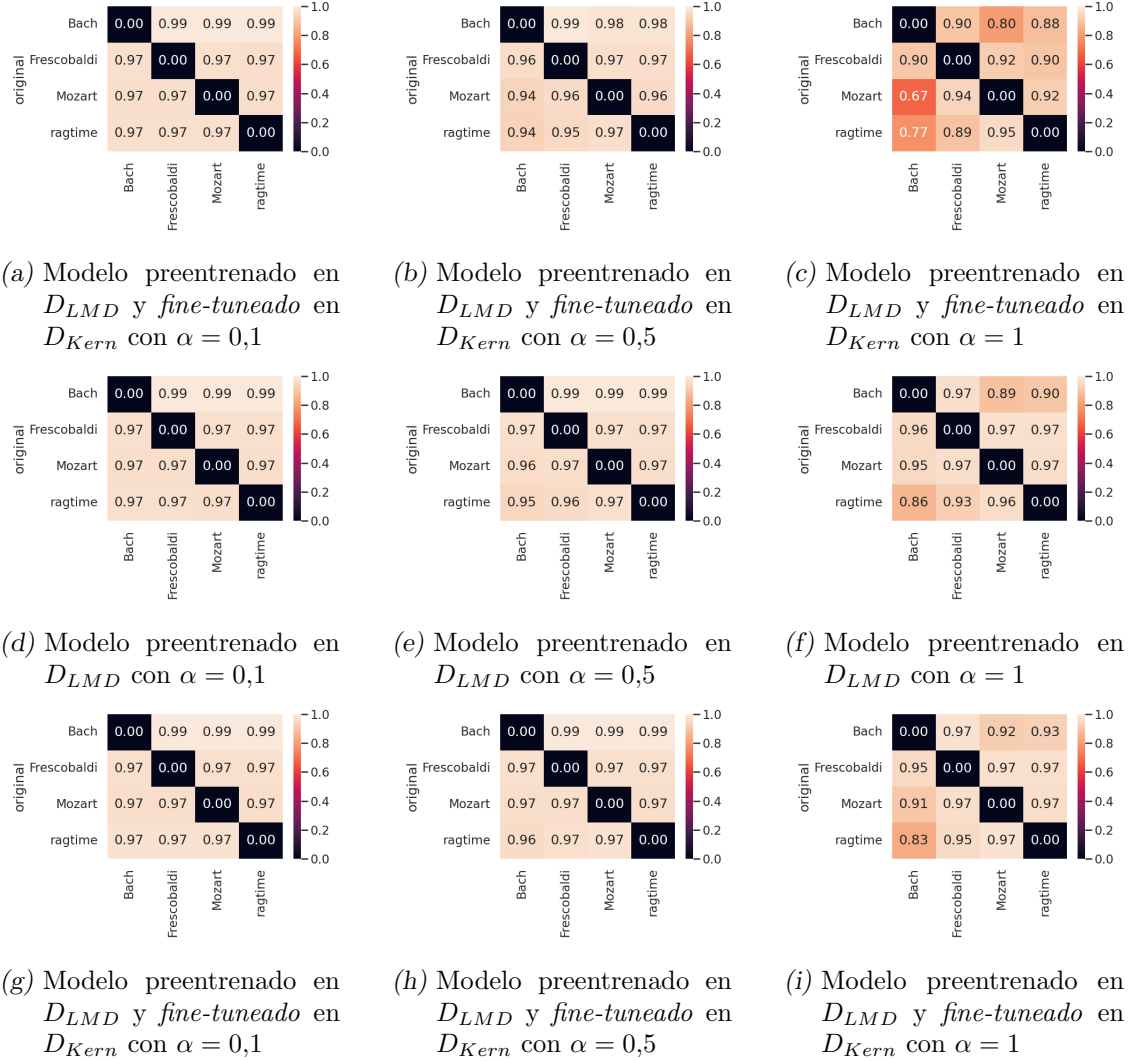


Fig. 5.7: Comparación de resultados de evaluar la función de fidelidad con el método de similitud de la *diferencia* con el método de *suma y resta* para el modelo *pre fine-tuning* y luego para el *post fine-tuning*. Los ejes y muestran los distintos s y los ejes x los distintos s' de cada transformación. Los valores corresponden a los promedios de evaluar el método de la *diferencia* en los fragmentos transformados de la transferencia correspondiente.

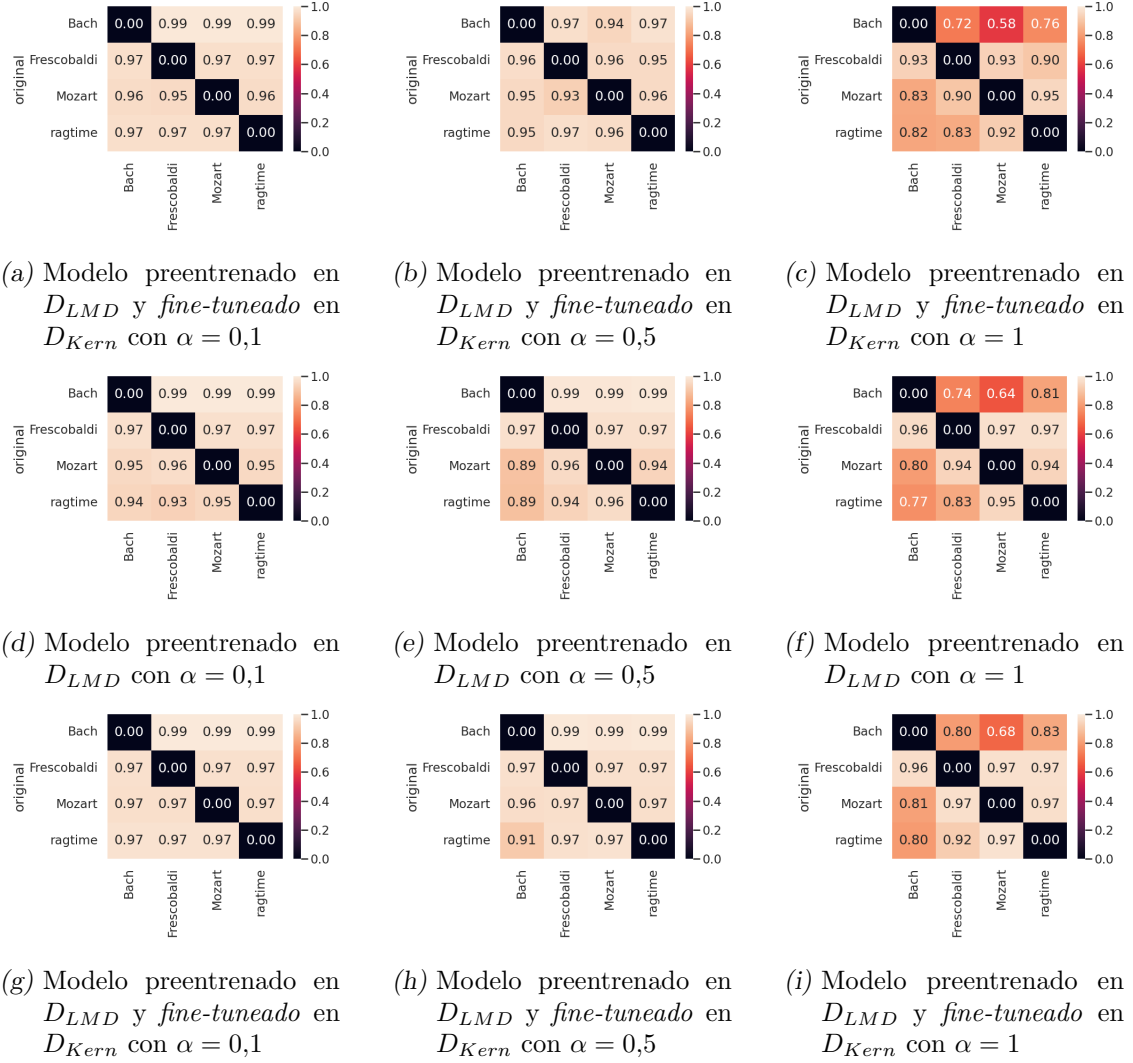


Fig. 5.8: Comparación de resultados de evaluar la función de fidelidad con el método de similitud de la *distancia* con el método de *suma y resta* para el modelo *pre fine-tuning* y luego para el *post fine-tuning*. Los ejes *y* muestran los distintos *s* y los ejes *x* los distintos *s'* de cada transformación. Los valores corresponden a los promedios de evaluar el método de la *distancia* en los fragmentos transformados de la transferencia correspondiente.

6. CONCLUSIONES

En el presente trabajo se ha buscado resolver el problema de transferencia de estilos en música, es decir, dado un fragmento musical de cierto estilo, representarlo en otro, manteniendo características que evoquen al original pero que también tenga características del nuevo estilo. Para ello se entrenaron tres VAEs a partir de fragmentos de 4 compases de piezas de cuatro estilos distintos: uno sobre un *dataset* chico de música que contiene 4 estilos que consideramos de interés (ver explicación en 2.3), otro sobre un *dataset* 40 veces más grande con música de otros estilos y un tercero aplicando *fine-tuning* a este último modelo sobre el primer *dataset*. Estos modelos fueron entrenados para reconstruir la entrada original, a la manera de una función identidad pero codificando la información en un espacio latente de menor dimensionalidad. Por su arquitectura, se puede operar en dicho espacio para transformar un fragmento de uno de los estilos por otro de ellos (ver explicación en 4.2).

Luego, se propusieron una serie de métricas para evaluar automáticamente los fragmentos generados por los tres modelos. Presentamos una metodología de evaluación para la tarea de transferencia de estilos en música basada en tres pilares: musicalidad, acercamiento al nuevo estilo y similitud con el fragmento musical original.

A continuación se escucharon algunos ejemplos tomados al azar para tener una primera aproximación de los resultados antes de evaluar con los métodos propuestos. En las escuchas no es muy evidente mayor acercamiento a los estilos cuando el parámetro α (que guía a la transformación) es grande sino una mayor deformación que no necesariamente implica acercamiento. Por otro lado, se pudo observar que las evaluaciones propuestas reflejan las apreciaciones que se tuvieron al escuchar una selección de los fragmentos generados. En primer lugar, los tres modelos lograron los objetivos buscados pero con distintas magnitudes: transformar el fragmento original con musicalidad para que se asemeje al nuevo estilo pero sin perder la identidad original. Sin embargo, cuanto mayor es la transformación, menor es la musicalidad del fragmento generado. En este sentido, el parámetro α que indica cuánto transformar el fragmento resulta de vital importancia para regular la transformación.

Además, la musicalidad y la similitud con el original se van perdiendo a medida que la transformación es mayor, pero por otro lado, en general se acercan cada vez más al nuevo estilo a medida que crece la magnitud de la transformación. Para ello, nuevamente es importante la graduación que se haga sobre el parámetro α para balancear el cambio del estilo con la pérdida de musicalidad que parece implicar un α más grande. En particular, comparamos dos metodologías para transformar: sumando el vector característico del nuevo estilo (método de la *suma*, ver ecuación 4.1) y sumando este vector y restando a su vez el vector característico del estilo del fragmento original (método de *suma y resta*, ver ecuación 4.2). Notamos que es importante el método que se use para transformar pues, por ejemplo, el método de la *suma* funciona mejor para transferencias con Bach como estilo original mientras que el de la *suma y resta* funciona mejor para transformar al estilo Frescobaldi con $\alpha = 0,5$.

Por otro lado, se evidencian diferencias entre el modelo *base* y los otros dos modelos. La performance del primero es peor en los tres ejes evaluados sugiriendo que es mejor entrenar con más datos no específicos que entrenar sobre los estilos en sí que queremos

transformar. Sin embargo, entre los dos últimos modelos no se ha notado mucha diferencia por lo que parece no ser necesario aplicar *fine-tuning*. Este proceso logra mejoras leves en algunas transformaciones específicas pero en otras incluso las dificulta.

Asimismo, las transformaciones no son parejas entre pares de estilos. Esto podría deberse a la diferencia o similitud entre ellos. Por ejemplo, los estilos más complejos rítmica y melódicamente (Mozart y ragtime) son los que peores métricas presentan cuando se quiere transformar entre sí. Con los otros estilos se obtienen mejores resultados posiblemente debido a que la diferencia de complejidad que presentan hace más evidente la transformación. Asimismo, son los que mayor musicalidad presentan cuando se los usa como estilo de partida, haciendo que posiblemente sea difícil desprenderse de ciertos rasgos que presenta la distribución de musicalidad M^{mus} . A su vez, parece resultar dificultosa la conservación de musicalidad al partir de estilos más simples a estos más complejos. Esto muestra la importancia de evaluar en estilos con diferentes grados de parecido como explicamos en 2.3.

6.1. Trabajo futuro

Notamos que mantener musicalidad en los fragmentos generados por los modelos no es trivial. A α grande, aunque puede aproximarse al nuevo estilo, la musicalidad se degrada. Esto podría mejorarse. Por ejemplo, podría utilizarse un *dataset* más grande de entrenamiento o cambiar la arquitectura de la red. Una posibilidad sería agregar una capa de LSTM bidireccional como en el *encoder* de MusicVAE [29].

Además, proponemos evaluar los audios producidos con otros oyentes como suele hacerse en la bibliografía [21, 25, 29]. En particular, esta metodología podría usarse para validar las métricas propuestas a partir de oyentes que otorguen su apreciación para nuestras tres métricas de interés (musicalidad, transformación y similitud al original). Por otro lado, se propone usar alguna métrica más robusta para evaluar la similitud entre el fragmento transformado y el original dado que no estamos considerando correctamente diferencias que podrían no ser tales desde la percepción musical (por ejemplo, si la diferencia entre ambos es solo un corrimiento entre alguno de los ejes - instante de tiempo o altura de la nota). Una idea posible es usar métricas de plagio [7]. Estas métricas se utilizan para detectar cuando dos piezas musicales pueden ser consideradas versiones de una misma idea musical. En particular, estos desarrollos tienen un objetivo detectar similitudes definidas bajo conceptos legales de plagio. Sin embargo, el trabajo realizado en esta área podría ser usado como base para robustecer la métrica de similitud en un trabajo futuro.

Por otro lado, dado que el presente trabajo surgió en base a continuar el *Requiem* K. 626 de Mozart, restaría poder entrenar algún modelo para esta tarea específica planteada originalmente.

Finalmente, dado que el presente trabajo comenzó antes del advenimiento de los *Large Language Models* como MusicLM [1] y su buena performance para distintas tareas, sería necesario probar el desempeño en esta tarea con dichos métodos.

BIBLIOGRAFÍA

- [1] Andrea Agostinelli et al. “Musiclm: Generating music from text”. En: *arXiv preprint arXiv:2301.11325* (2023).
- [2] Rongjie Huang et al. “AudioGPT: Understanding and Generating Speech, Music, Sound, and Talking Head”. En: *arXiv:2304.12995* (2023).
- [3] Christodoulos Benetatos, Joseph VanderStel y Zhiyao Duan. “Bachduet: A deep learning system for human-machine counterpoint improvisation”. En: *Proceedings of the International Conference on New Interfaces for Musical Expression* (2020).
- [4] Jacopo de Berardinis et al. “Modelling long- and short-term structure in symbolic music with attention and recurrence”. En: *Joint Conference on AI Music Creativity* (2020).
- [5] Vance W. Berger y YanYan Zhou. “Kolmogorov–Smirnov Test: Overview”. En: *Wiley StatsRef: Statistics Reference Online*. John Wiley Sons, Ltd, 2014. ISBN: 9781118445112. DOI: <https://doi.org/10.1002/9781118445112.stat06558>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781118445112.stat06558>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781118445112.stat06558>.
- [6] Jean-Pierre Briot, Gaëtan Hadjeres y François-David Pachet. *Deep learning techniques for music generation*. Vol. 1. Springer, 2020.
- [7] Robert J.S. Cason y Daniel Müllensiefen. “Singing from the same sheet: computational melodic similarity measurement and copyright law”. En: *International Review of Law, Computers & Technology* 26.1 (2012), págs. 25-36. DOI: 10.1080/13600869.2012.646786.
- [8] N. Chehebar. “Transporte Óptimo y Baricentro con distancia de Fermat”. 2021.
- [9] Rewon Child et al. *Generating Long Sequences with Sparse Transformers*. 2019. arXiv: 1904.10509 [cs.LG].
- [10] Rémi Flamary et al. “POT: Python Optimal Transport”. En: *Journal of Machine Learning Research* 22.78 (2021), págs. 1-8. URL: <http://jmlr.org/papers/v22/20-451.html>.
- [11] Leon A. Gatys, Alexander S. Ecker y Matthias Bethge. “A neural algorithm of artistic style”. En: *arXiv:1508.06576v2* (2015).
- [12] Robert Gauldin. *Harmonic Practice in Tonal Music*. W. W. Norton, 2004.
- [13] Rui Guo et al. “A variational autoencoder for music generation controlled by tonal tension”. En: oct. de 2020.
- [14] Susan Hallam. *Musicality*. Oxford University Press, 2006.
- [15] Xianxu Hou et al. “Deep Feature Consistent Variational Autoencoder”. En: *2017 IEEE Winter Conference on Applications of Computer Vision (WACV)*. 2017, págs. 1133-1141. DOI: 10.1109/WACV.2017.131.

-
- [16] Yongcheng Jing et al. “Neural Style Transfer: A Review”. En: *IEEE Transactions on Visualization and Computer Graphics* 26.11 (2020), págs. 3365-3385. DOI: 10.1109/TVCG.2019.2921336.
- [17] Daniel D. Johnson. “Generating Polyphonic Music Using Tied Parallel Networks”. En: *Computational Intelligence in Music, Sound, Art and Design*. Ed. por João Correia, Vic Ciesielski y Antonios Liapis. Cham: Springer International Publishing, 2017, págs. 128-143. ISBN: 978-3-319-55750-2.
- [18] Diederik P. Kingma y Max Welling. “Auto-Encoding Variational Bayes”. En: *CoRR* abs/1312.6114 (2013). URL: <https://api.semanticscholar.org/CorpusID:216078090>.
- [19] Qiuqiang Kong et al. *GiantMIDI-Piano: A large-scale MIDI dataset for classical piano music*. 2022. arXiv: 2010.07061 [cs.IR].
- [20] Dimos Makris et al. “Combining LSTM and Feed Forward Neural Networks for Conditional Rhythm Composition”. En: *International Conference on Engineering Applications of Neural Networks*. 2017. URL: <https://api.semanticscholar.org/CorpusID:41623914>.
- [21] Huanru Henry Mao, Taylor Shin y Garrison Cottrell. “DeepJ: Style-specific music generation”. En: *2018 IEEE 12th International Conference on Semantic Computing (ICSC)*. IEEE. 2018, págs. 377-382.
- [22] Midjourney. *Midjourney*. 2022. URL: <https://www.midjourney.com/home>.
- [23] Tomas Mikolov et al. “Distributed Representations of Words and Phrases and their Compositionality”. En: *Advances in Neural Information Processing Systems* 26 (oct. de 2013).
- [24] OpenAI. *DALL-E 2*. 2022. URL: <https://openai.com/dall-e-2>.
- [25] Christine Payne. “MuseNet”. En: *OpenAI Blog* 3 (2019).
- [26] Gabriel Peyré y Marco Cuturi. *Computational Optimal Transport*. 2020. arXiv: 1803.00567 [stat.ML].
- [27] Lucas Pinheiro Cinelli et al. *Variational Methods for Machine Learning with Applications to Deep Networks*. Springer, 2021.
- [28] C. Raffel. “Learning-Based Methods for Comparing Sequences, with Applications to Audio-to-MIDI Alignment and Matching”. 2016.
- [29] Adam Roberts et al. “A Hierarchical Latent Vector Model for Learning Long-Term Structure in Music”. En: *Proceedings of the 35th International Conference on Machine Learning* abs/1803.05428 (2018). arXiv: 1803.05428. URL: <http://arxiv.org/abs/1803.05428>.
- [30] P. Rodríguez Zivic, F. Shifres y G. Cecchi. “Perceptual basis of evolving Western musical styles”. En: *Proceedings of the National Academy of Sciences of the United States of America* 110 (2013).
- [31] C.E. Shannon. “A Mathematical Theory of Communication”. En: *The Bell System Technical Journal* 27 (1948).
- [32] *Stability AI*. Accedido el 22/10/2023. URL: <https://stability.ai/>.

-
- [33] Felix Sun. *DeepHear – Composing and harmonizing music with neural networks*. Accedido el 22/10/2023. URL: <https://fephsun.github.io/2015/09/01/neural-music.html>.
 - [34] Christian Walder. *Symbolic Music Data Version 1.0*. 2016. arXiv: 1606.02542 [cs.SD].
 - [35] Shangda Wu y Maosong Sun. “Exploring the Efficacy of Pre-trained Checkpoints in Text-to-Music Generation Task”. En: *The AAAI-23 Workshop on Creative AI Across Modalities*. 2023. URL: <https://openreview.net/forum?id=QmWXskBhesn>.
 - [36] Li-Chia Yang, Szu-Yu Chou y yi-hsuan Yang. “MidiNet: A Convolutional Generative Adversarial Network for Symbolic-domain Music Generation using 1D and 2D Conditions”. En: (mar. de 2017).